



Database System

**Iqra Batool
Mansoor-e-Zahra
Takreem Atif**

CIS(23 – 27)

Project Report : Food Delivery App

Submitted to : Dr Irfan Hameed

Submission Date : December 9, 2025

Table of Contents

<i>On-Demand Food Delivery Service Database</i>	4
<i>Introduction:</i>	4
<i>Project Description :</i>	5
Key Stakeholders.....	5
Core Business Workflow	5
Business Rules	5
<i>Enterprise Data Model:</i>	6
Entities	6
Relationships	6
1. Order — Payment Relationship	6
2. Rider — Order Relationship	7
3. Restaurant — Order Relationship.....	7
4. Restaurant — MenuItem Relationship	7
5. Order — OrderItem Relationship	8
6. MenuItem — OrderItem Relationship	8
<i>Entity Relationship Model:</i>	10
Entities & Attributes	10
Relationships & Cardinality	12
1) Customer and Order	12
2) Restaurant and Order	12
3) Rider and Order	13
4) Order and Payment	13
5) Restaurant and MenuItem (Product).....	14
6) Order and OrderItem	14
7) MenuItem and OrderItem	15
<i>Relational Model:</i>	17
Entities Tables	17
Relational Tables	19
Relational Schema	20
Integrity Constraints	20
Final Relational Schema	22
<i>Normalization:</i>	23
1. First Normal Form (1NF).....	23

2. Second Normal Form (2NF)	24
3. Third Normal Form (3NF)	25
Normalization Scenario	27
Normalizing to Second Normal Form (2NF)	27
Normalizing to Third Normal Form (3NF)	28
SQL Code	30
Database Setup	30
TABLES	30
Data Insertion	33
Users, Roles, and Access Control (DCL)	41
TRIGGERS	45
Transactions	54
ROLES	58

On-Demand Food Delivery Service Database

Introduction:

The *On-Demand Food Delivery Service Database* project focuses on designing and implementing a structured and reliable Relational Database Management System (RDBMS) to support the complete operational workflow of an online food-delivery platform. The goal is to build a database that accurately manages customers, restaurants, menu offerings, orders, riders, and payment transactions, while ensuring data integrity, consistency, and secure access. Using SQL Server, the project emphasizes strong schema design, proper normalization, enforcement of referential integrity, and implementation of transactional rules that reflect real-world business operations.

The database serves as the central backbone that enables the platform to process orders efficiently—from menu browsing and ordering to dispatch, delivery, and payment settlement.

Project Description :

Key Stakeholders

The system is built around coordinating these five parties:

1. **Customer:** Places orders, pays, and submits reviews.
2. **Restaurant (Vendor):** Accepts, prepares, and confirms order fulfillment.
3. **Rider (Delivery Agent):** Picks up orders and completes the delivery.
4. **Admin / Platform:** Manages the entire operation, user onboarding, finances, and customer support.
5. **Payment Gateway:** Handles all financial transactions (Card, Online, or COD settlement).

Core Business Workflow

The process follows a logical sequence centered on the **Order** entity:

1. **Order Placement:** The **Customer** selects a menu, chooses a payment method (stored in the **Payment** table), and places the order.
2. **Preparation:** The system sends the order to the **Restaurant**, which accepts it and updates the status to "**Preparing.**"
3. **Dispatch:** As the food nears completion, the platform locates an available **Rider** and assigns the delivery job.
4. **Transit:** The **Rider** picks up the order ("Picked Up") and delivers it to the customer ("Delivered").
5. **Settlement:** The **Payment** is confirmed (via Gateway or COD collection), and the platform distributes funds to the relevant parties.
6. **Feedback:** The **Customer** finalizes the loop by reviewing the service.

Business Rules

1. An Order must contain at least one Menu Item.
2. A Customer can place many Orders.
3. A Restaurant must offer one or many Menu Items.
4. A Menu Item must be offered by one and only one Restaurant.
5. An Order must be delivered by exactly one Rider (once assigned).
6. An Order is linked to exactly one Payment.
7. A Payment is linked to exactly one Order.
8. Online payments are preferred, and a bill can be generated even if the order is not placed at the restaurant.
9. An Order must be placed by one and only one Customer.
10. An Order must be fulfilled by one and only one Restaurant.
11. A Rider can deliver many Orders.
12. An Order must contain one or many Order Items (line items).

Enterprise Data Model:

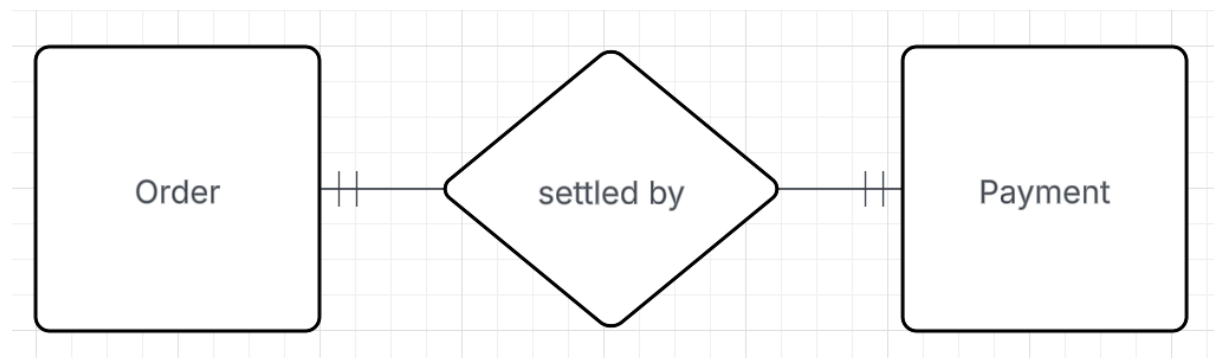
Entities

- Customer
- Restaurant
- Rider
- Order
- MenuItem
- Payment

Relationships

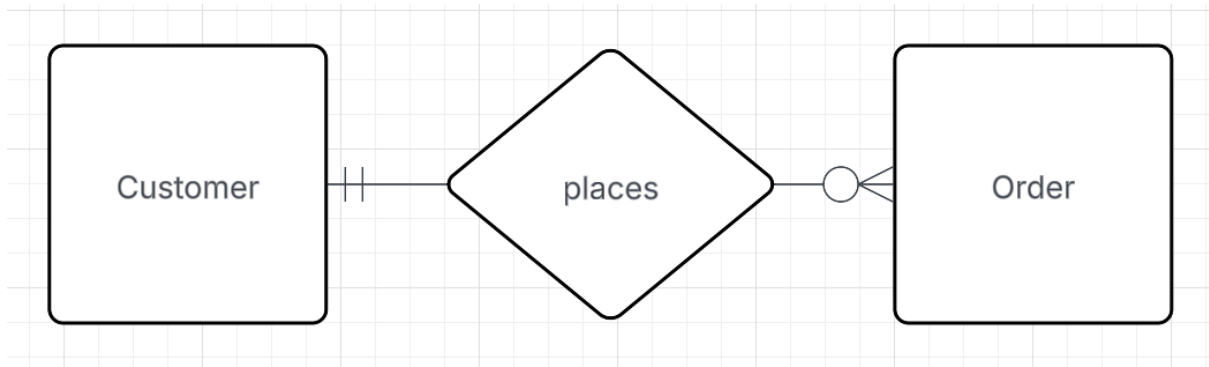
1. Order — Payment Relationship

Payments are created for orders to record how each transaction is **financially settled**.



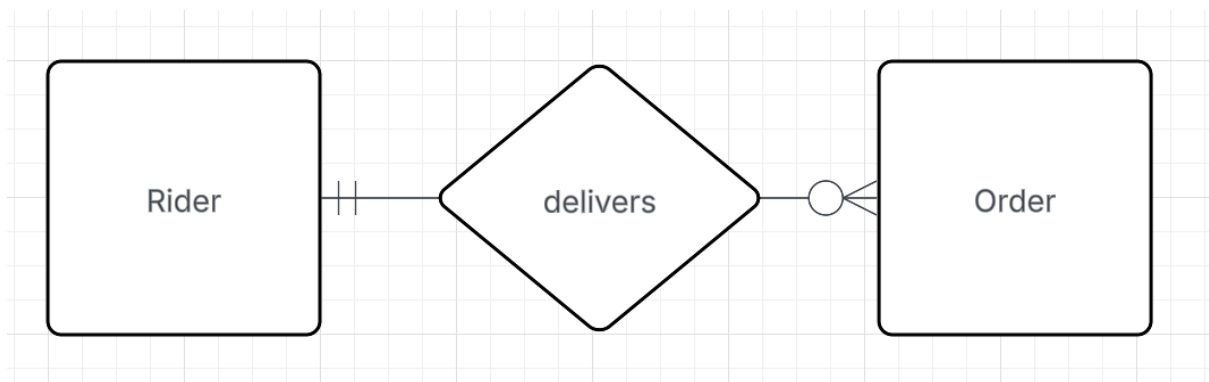
1) Customer — Order Relationship

Customers interact with the system by **placing** food orders.



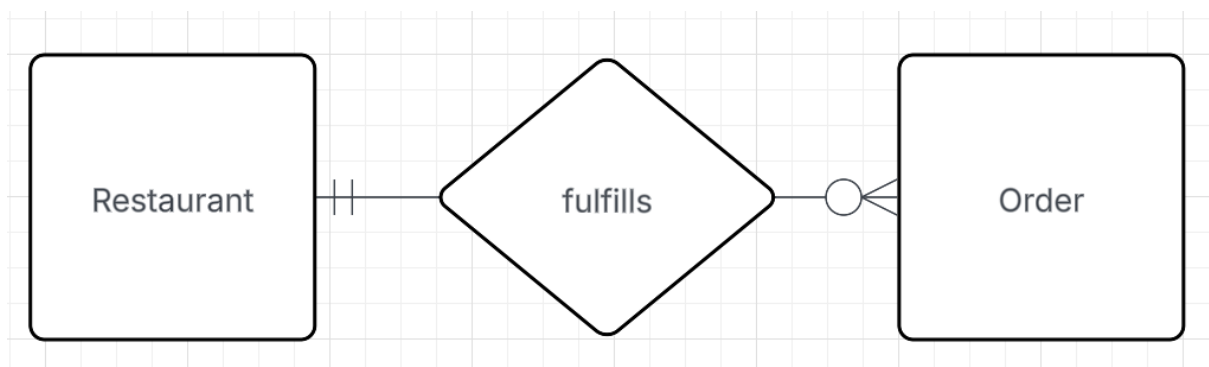
2. Rider — Order Relationship

Riders are responsible for **transporting** prepared orders from restaurants to customers.



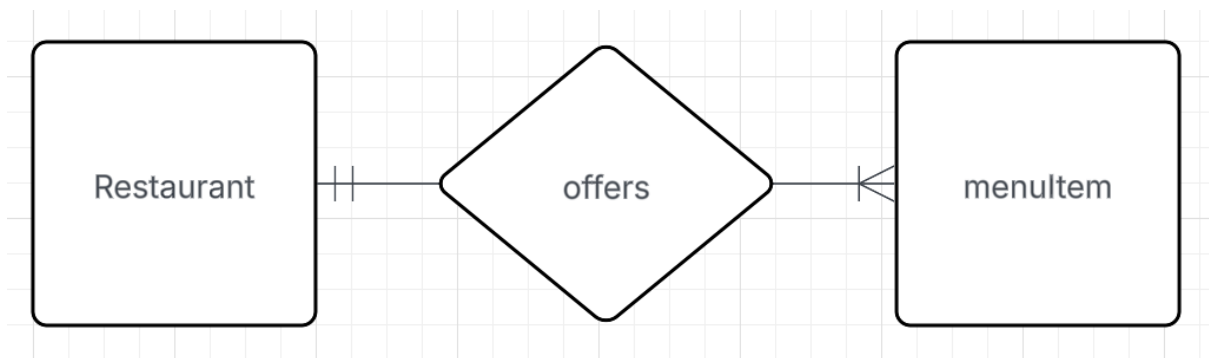
3. Restaurant — Order Relationship

Restaurants **receive** and **fulfill** orders submitted by customers.



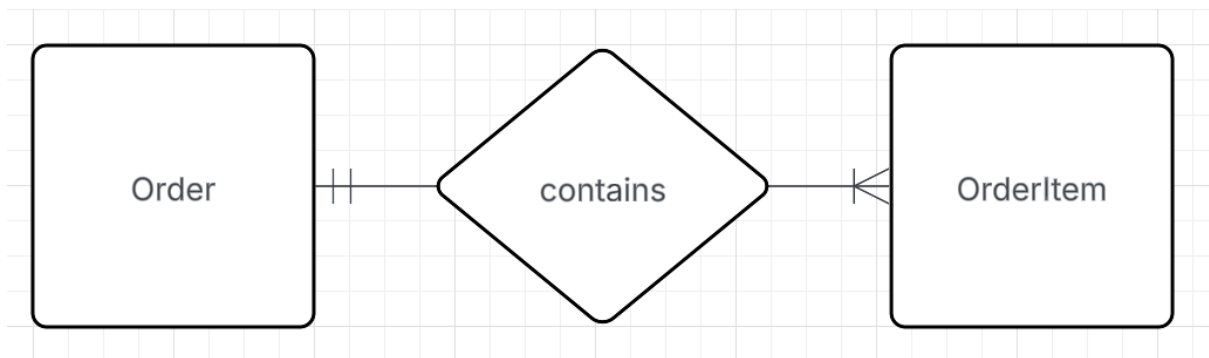
4. Restaurant — MenuItem Relationship

Restaurants **offer** a variety of menu items to customers.



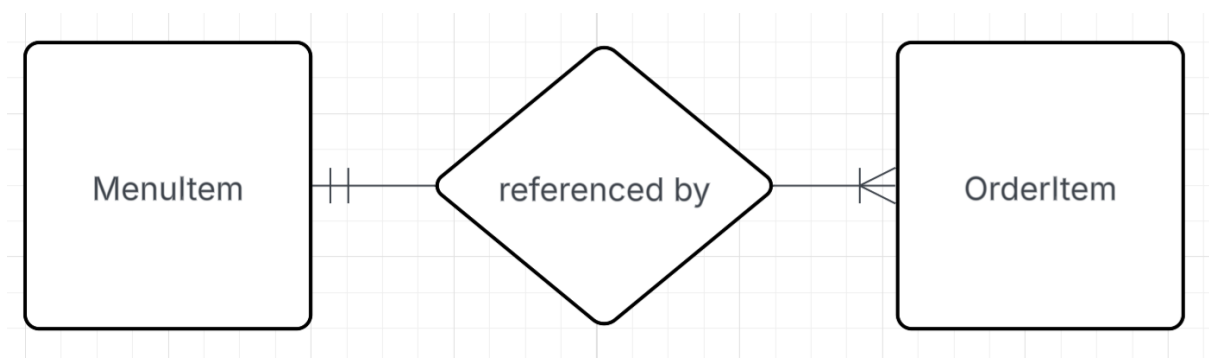
5. Order — OrderItem Relationship

Orders **consist** of multiple items selected by customers.

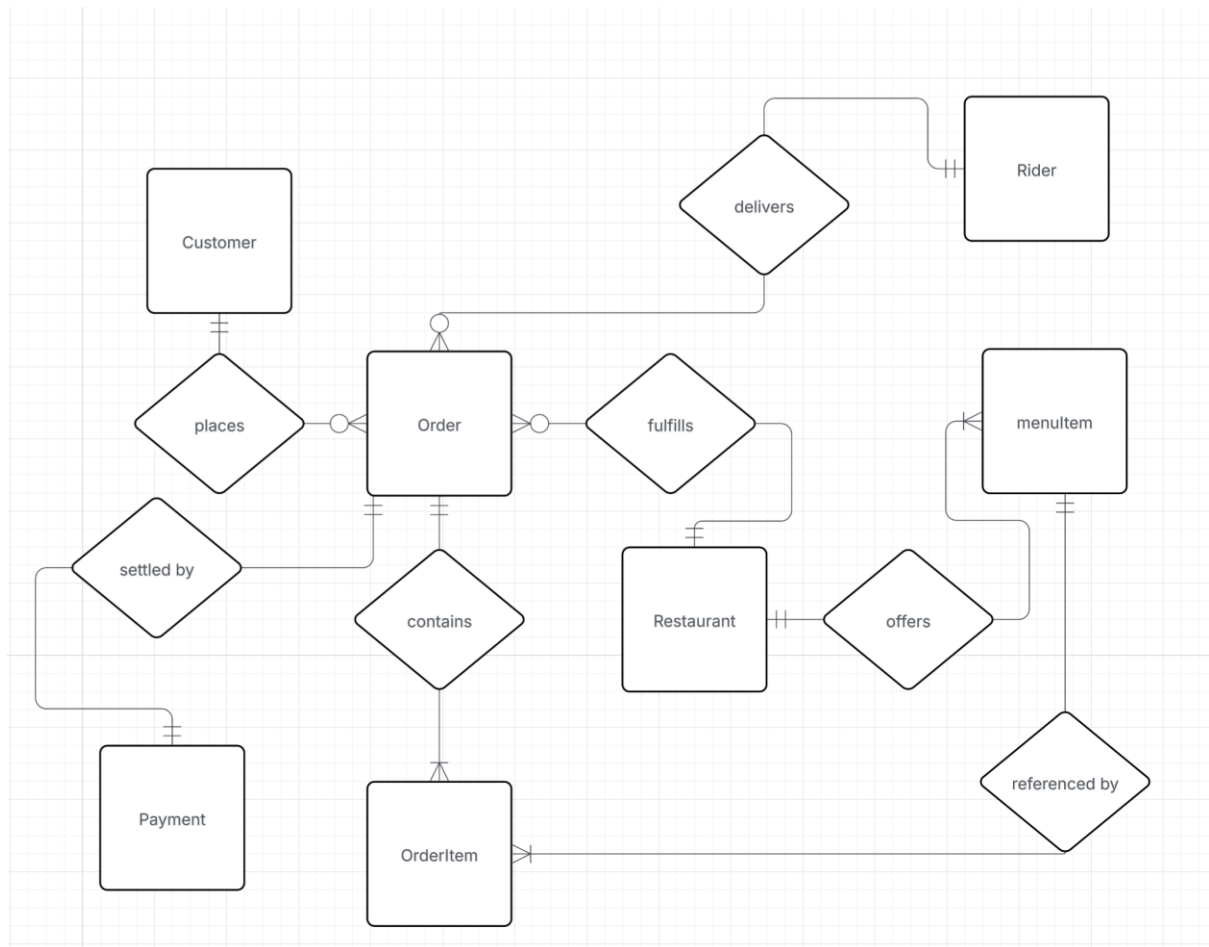


6. MenuItem — OrderItem Relationship

Each order item **corresponds** to a specific menu item offered by a restaurant.

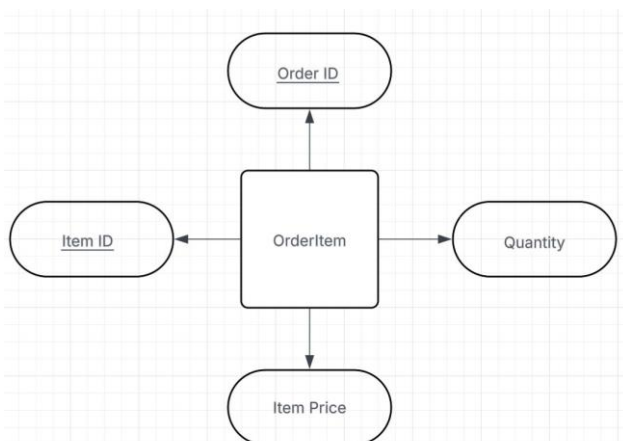
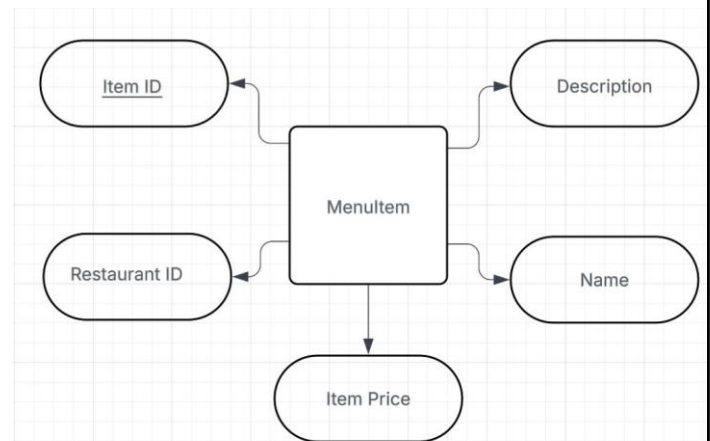
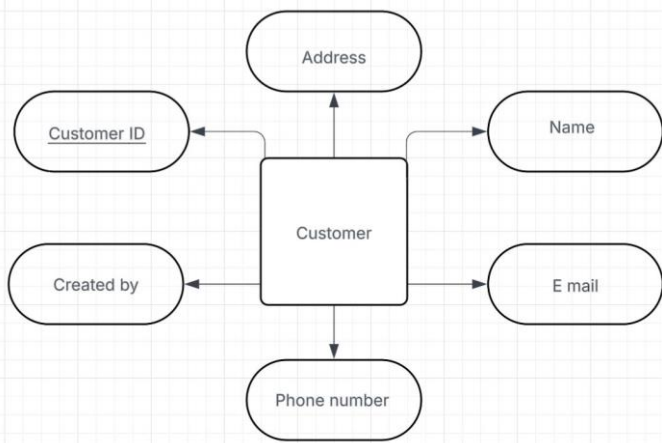


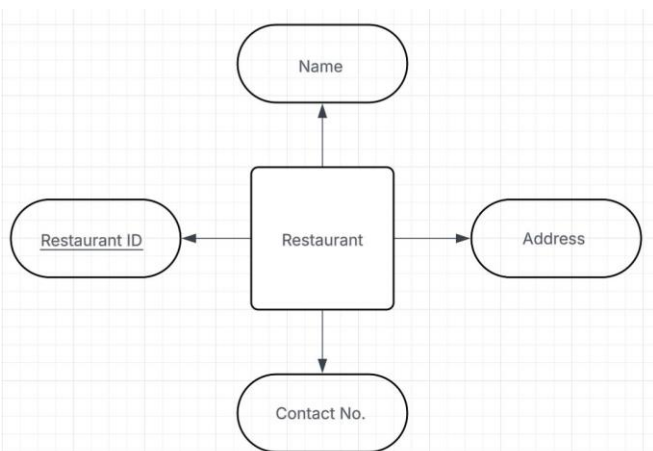
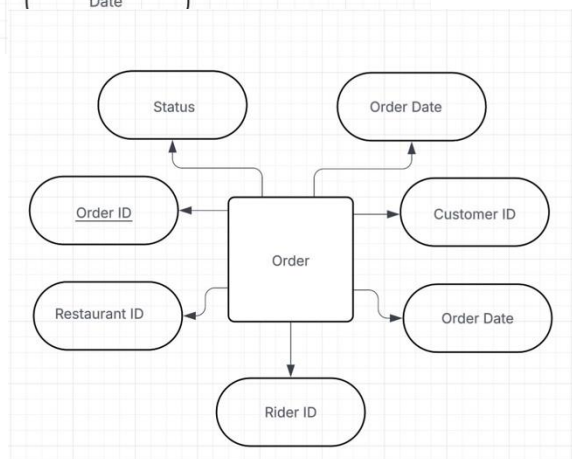
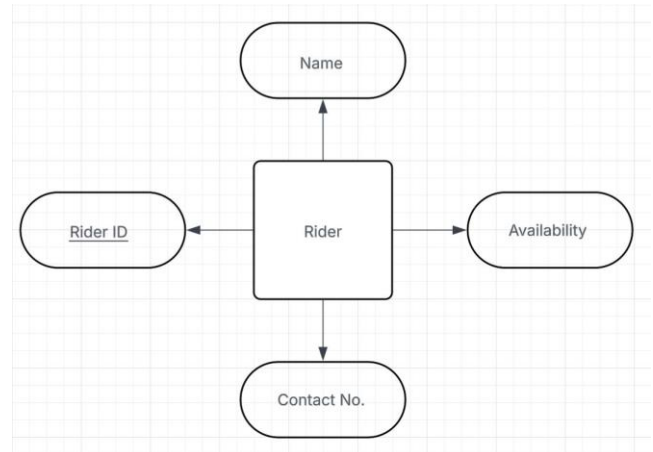
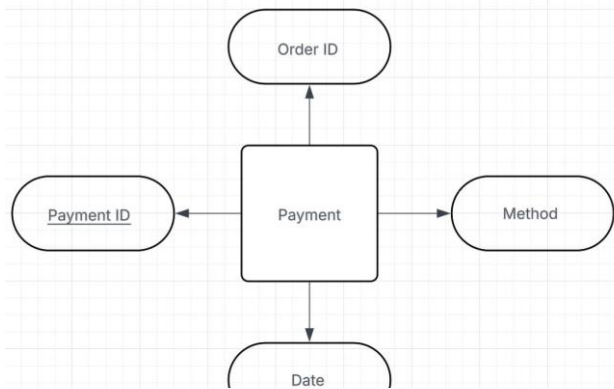
Final EDM



Entity Relationship Model:

Entities & Attributes



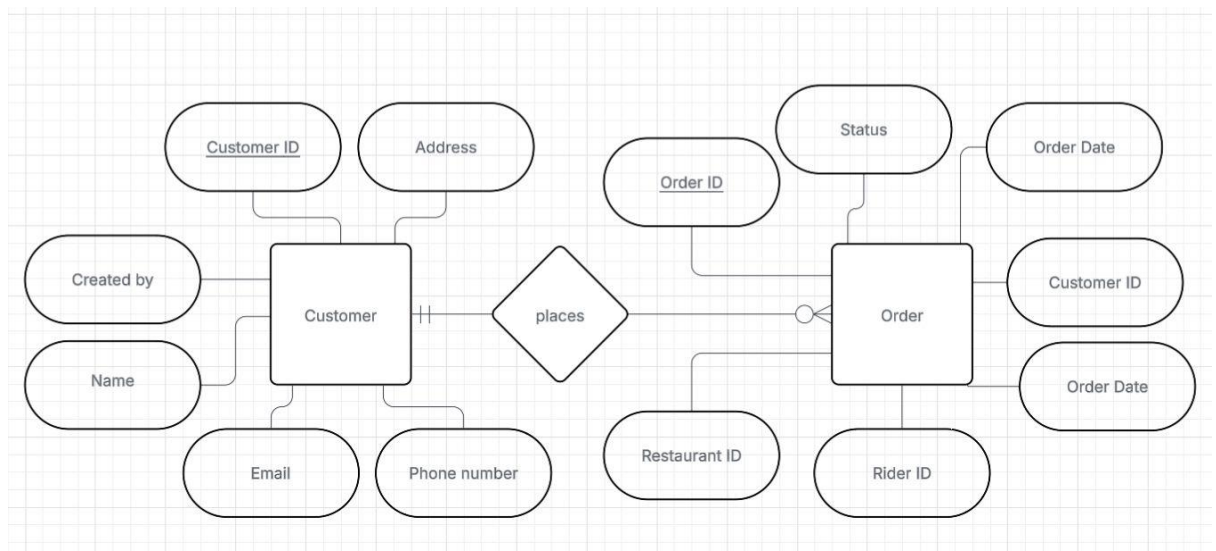


Relationships & Cardinality

1) Customer and Order

Customer - Places - Order

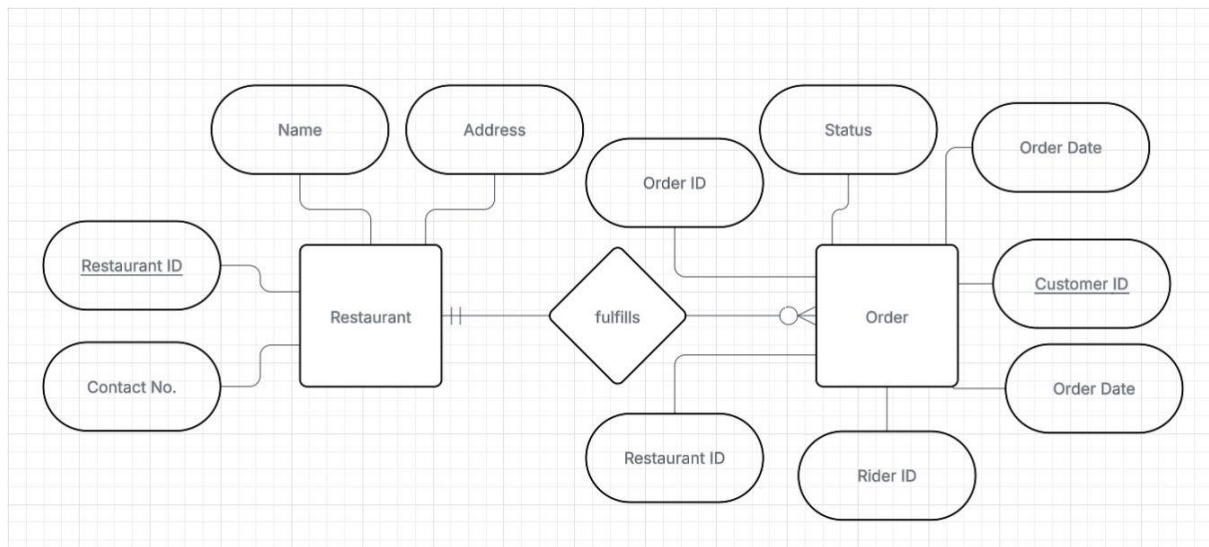
- A **customer** can place **zero or many** orders.
- An **order** must be placed by **one and only one** customer.



2) Restaurant and Order

Restaurant - Fulfills - Order

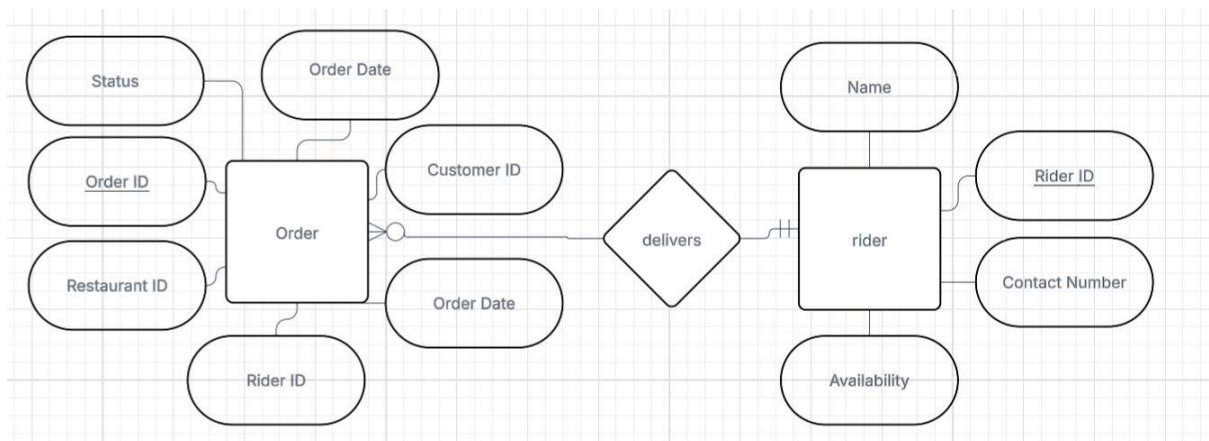
- A **restaurant** must fulfill **zero or many** orders.
- An **order** must be fulfilled by **one and only one** restaurant.



3) Rider and Order

Rider - Delivers - Order

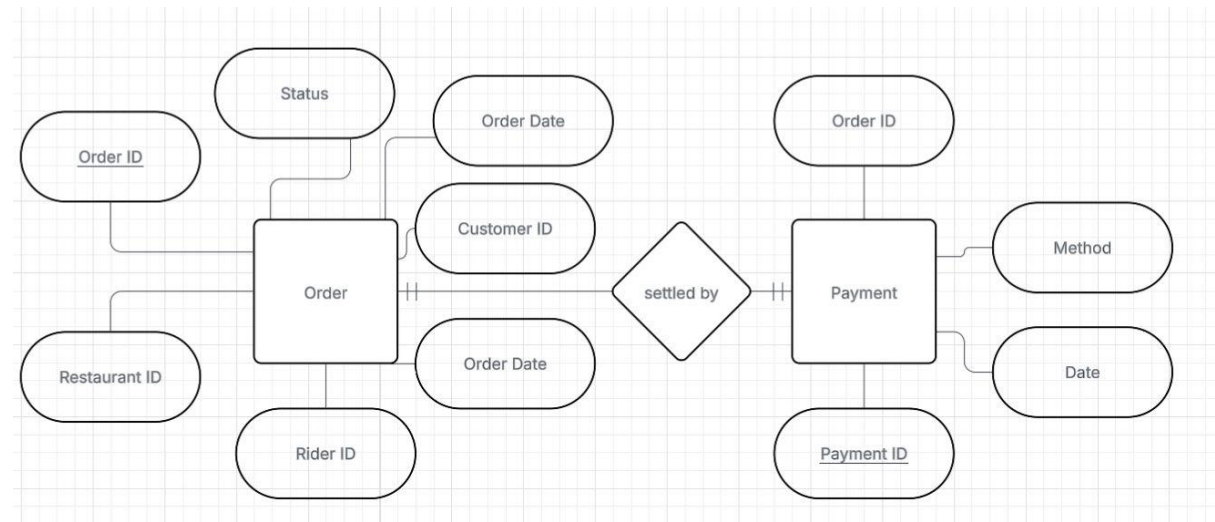
- A **rider** can deliver **zero or many** orders.
- An **order** must be delivered by **one and only one** rider (once assigned).



4) Order and Payment

Order - Settled By - Payment

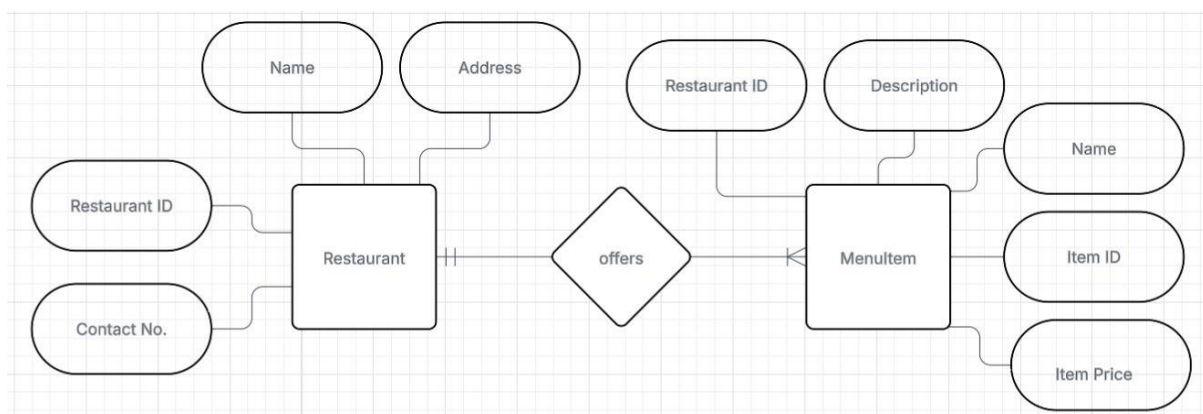
- An **order** is settled by **one and only one** payment.
- A **payment** settles **one and only one** order.



5) Restaurant and MenuItem (Product)

Restaurant - Offers - MenuItem

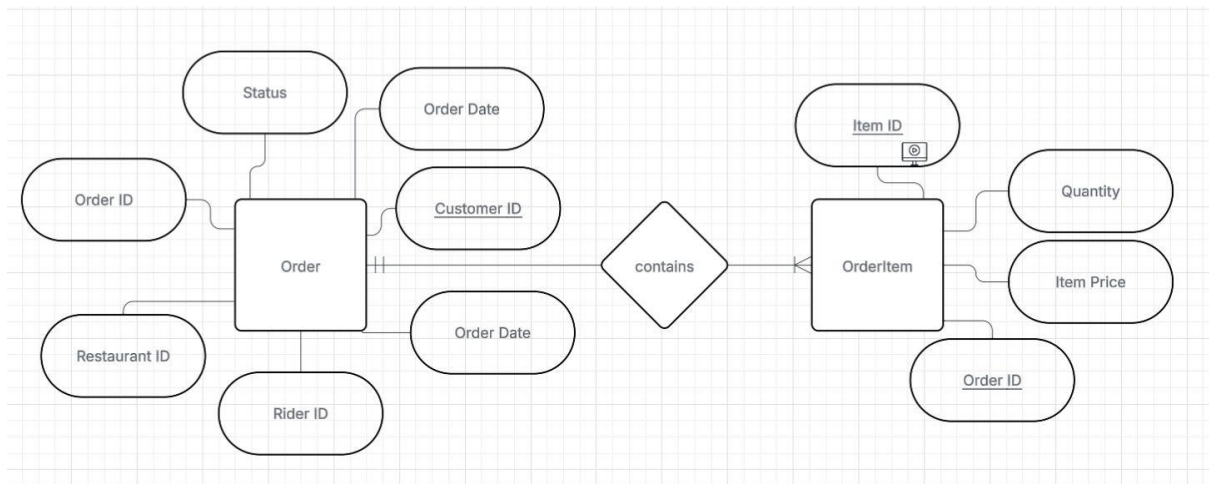
- A **restaurant** must offer **one or many** menu items.
- A **menu item** must be offered by **one and only one** restaurant.



6) Order and OrderItem

Order - Contains - OrderItem

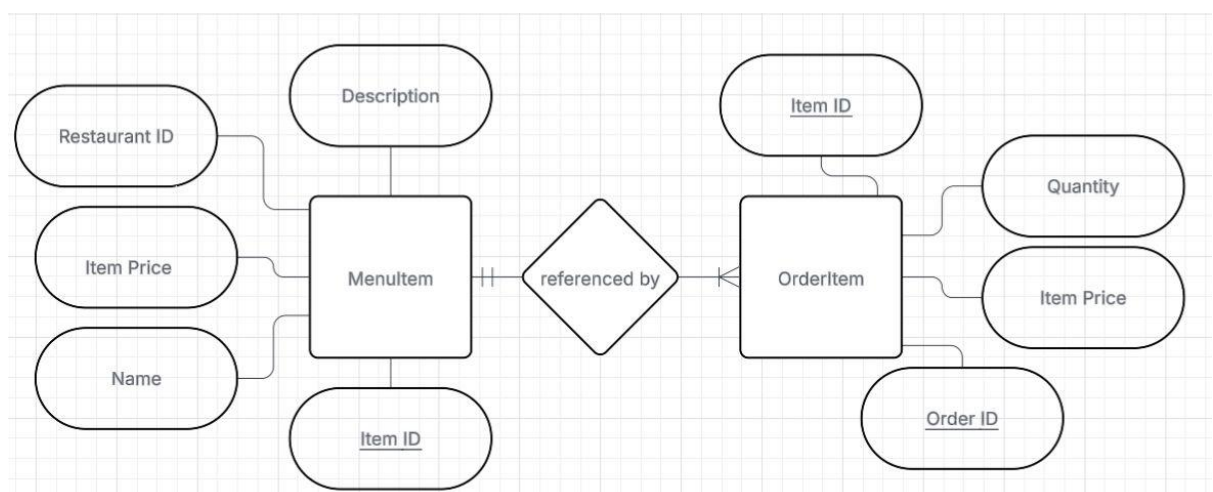
- An **order** must contain **one or many** order items (line items).
- An **order item** belongs to **one and only one** order.



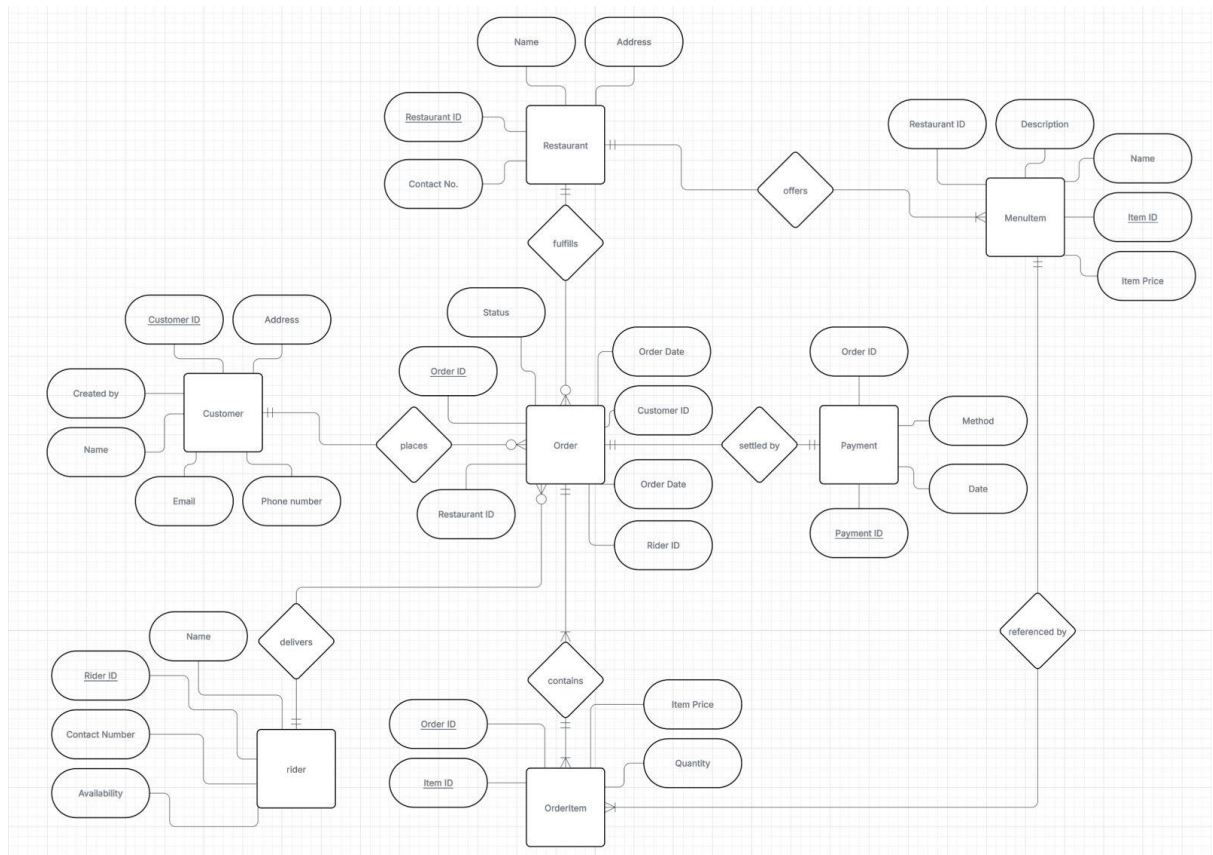
7) MenuItem and OrderItem

MenuItem - Referenced By - OrderItem

- A **menu item** must be referenced by **one or many** order items.
- An **order item** references **one and only one** menu item.



Final ER Diagram:



Description:

The most critical structural element in this ER model is the **OrderItem** entity, which functions as an **associative entity** to resolve the complex many-to-many relationship between **Order** and **MenuItem**. This entity interposes itself to create two manageable one-to-many relationships, making the system efficient and accurate. Its primary purpose is to hold **transaction-specific details** that belong to the purchase instance, not the item or the order alone. Specifically, it tracks:

- **Quantity:** How many of a single item were bought in that specific order.
- **Price at Time of Order (Historical Price):** The exact price paid, which prevents future menu price changes from corrupting past order records

Relational Model:

Entities Tables

1) Customer

Attributes	Optional/ Required	Single / Composite	Simple/ Multi value	Stored/ Derived	Notes
Customer_ID	Required	Single	Simple	Stored	Primary Key
Name	Required	Composite (first/last)	Simple	Stored	
E_mail	Required	Single	Simple	Stored	Unique
Phone_NO	Required	Single	Multi value	Stored	Can be multiple
Created By	Required	Single	Simple	Stored	Date created
Address	Required	Single	Simple	Stored	

2) Restaurant

Attributes	Optional/ Required	Single / Composite	Simple / Multi value	Stored/ Derived	Notes
Restaurant_ID	Required	Single	Simple	Stored	Primary Key
Name	Required	Single	Simple	Stored	
Address	Required	Single	Simple	Stored	
Contact NO	Required	Single	Simple	Stored	

3) MenuItem

Attributes	Optinal/ Required	Simple/ Composite	Simple / Multi value	Stored/ Derived	Notes
Item_ID(PK)	Required	Single	Simple	Stored	Primary Key
Restaurant_ID	Required	Single	Simple	Stored	Foreign Key
Name	Required	Single	Simple	Stored	
Description	Optional	Single	Simple	Stored	
Price	Required	Single	Simple	Stored	

4) Rider

Attributes	Optinal/ Required	Simple/ Composite	Simple / Multi value	Stored/ Derived	Notes
Rider_ID	Required	Single	Simple	Stored	Primary Key
Name	Required	Single	Simple	Stored	
Contact NO	Required	Single	Simple	Stored	
Availability	Required	Single	Simple	Stored	Boolean

5) Payment

Attributes	Optinal/ Required	Simple/ Composite	Simple / Multi value	Stored/ Derived	Notes
Payment_ID	Required	Single	Simple	Stored	Primary Key
Order_ID	Required	Single	Simple	Stored	Foreign Key
Method	Required	Single	Simple	Stored	
Date	Required	Single	Simple	Stored	

6) Order

Attributes	Optional/ Required	Simple/ Composite	Simple / Multi value	Stored/ Derived	Notes
Order_ID	Required	Single	Simple	Stored	Primary Key
Customer_ID	Required	Single	Simple	Stored	Foreign Key
Restaurant_ID	Required	Single	Simple	Stored	Foreign Key
Rider_ID	Optional	Single	Simple	Stored	Foreign Key
Order Date	Required	Single	Simple	Stored	
Total Amount	Required	Single	Simple	Derived	From Order
Status	Required	Single	Simple	Stored	

7) OrderItem(Associative Entity)

Attributes	Optinal/ Required	Simple/ Composite	Simple / Multi value	Stored/ Derived	Notes
Order_ID(PK)	Required	Single	Simple	Stored	Foreign Key
Item_ID(PK)	Required	Single	Simple	Stored	Foreign Key
Quantity	Optional	Single	Simple	Stored	
Item Price	Required	Single	Simple	Stored	

Relational Tables

CUSTOMER

	Customer
PK	<u>Customer_ID</u>
	Name
	E_mail
	Phone_NO
	Created By
	Address

RESTAURANT

	Restaurant
PK	<u>Restaurant_ID</u>
	Name
	Address
	Contact NO

PAYMENT

	Payment
PK	<u>Payment_ID</u>
	Order_ID
	Method
	Date

MENUITEM

	Menuitem
PK	<u>Item_ID</u>
	Restaurant_ID
	Name
	Description
	Price

ORDER

	Order
PK	<u>Order_ID</u>
FK	Customer_ID
FK	Restaurant_ID
FK	Rider_ID
	Order Date
	Total Amount
	Status

RIDER

	Rider
PK	<u>Rider_ID</u>
	Name
	Contact NO
	Availability

ORDERITEM

	OrderItem
PK	<u>Order_ID</u>
PK	<u>Item_ID</u>
	Quantity
	Item Price

Relational Schema

Customer

<u>Customer_ID</u>	First Name	Last Name	E_mail	Phone No	Created At
--------------------	------------	-----------	--------	----------	------------

Restaurant

<u>Restaurant_ID</u>	Name	Street	City	Contact No
----------------------	------	--------	------	------------

Menu Item

<u>Item_ID</u>	<u>Restaurant_ID</u>	Name	Description	Price	Available
----------------	----------------------	------	-------------	-------	-----------

Rider

<u>Rider_ID</u>	Name	Contact No	Availability
-----------------	------	------------	--------------

Order

<u>Order_ID</u>	<u>Customer_ID</u>	<u>Restaurant_ID</u>	<u>Rider_ID</u>	Order Date	Total Amount	Status
-----------------	--------------------	----------------------	-----------------	------------	--------------	--------

Payment

<u>Payment_ID</u>	<u>Order_ID</u>	Method	Amount	Payment Date	Status
-------------------	-----------------	--------	--------	--------------	--------

OrderItem

<u>Order_ID</u>	<u>Item_ID</u>	Quantity	Item price
-----------------	----------------	----------	------------

Integrity Constraints

- **Entity Integrity:**

Enforced by **PRIMARY KEY** (PK must be **UNIQUE** and **NOT NULL**).

In our online food delivery system all the primary key are NOT NULL and UNIQUE. Like...

Customer_ID

Restaurant_ID

Item_ID

Rider_ID

Order_ID

Payment_ID

Order_ID+ Item_ID

- **Domain Integrity:**

Domain Integrity ensures that the values within a column are valid and conform to specific formats and constraints.

Some Common Entities attributes domain constraints:-

Customer.Email -> NVARCHAR(100) UNIQUE

Customer.CreatedAt -> DATETIME DEFAULT GETDATE()

Customer.Contact No -> NVARCHAR(20)

MenuItem.Price -> DECIMAL(10,2) (Implied: CHECK Price > 0)

MenuItem.Available -> BIT

Order.Status -> NVARCHAR(30) Restricted Values (ENUM/CHECK)

Order.Order Date -> DATETIME DEFAULT GETDATE()

Restaurant.Contact Number -> NVARCHAR(20)

Restaurant.Contact No -> NVARCHAR(20)

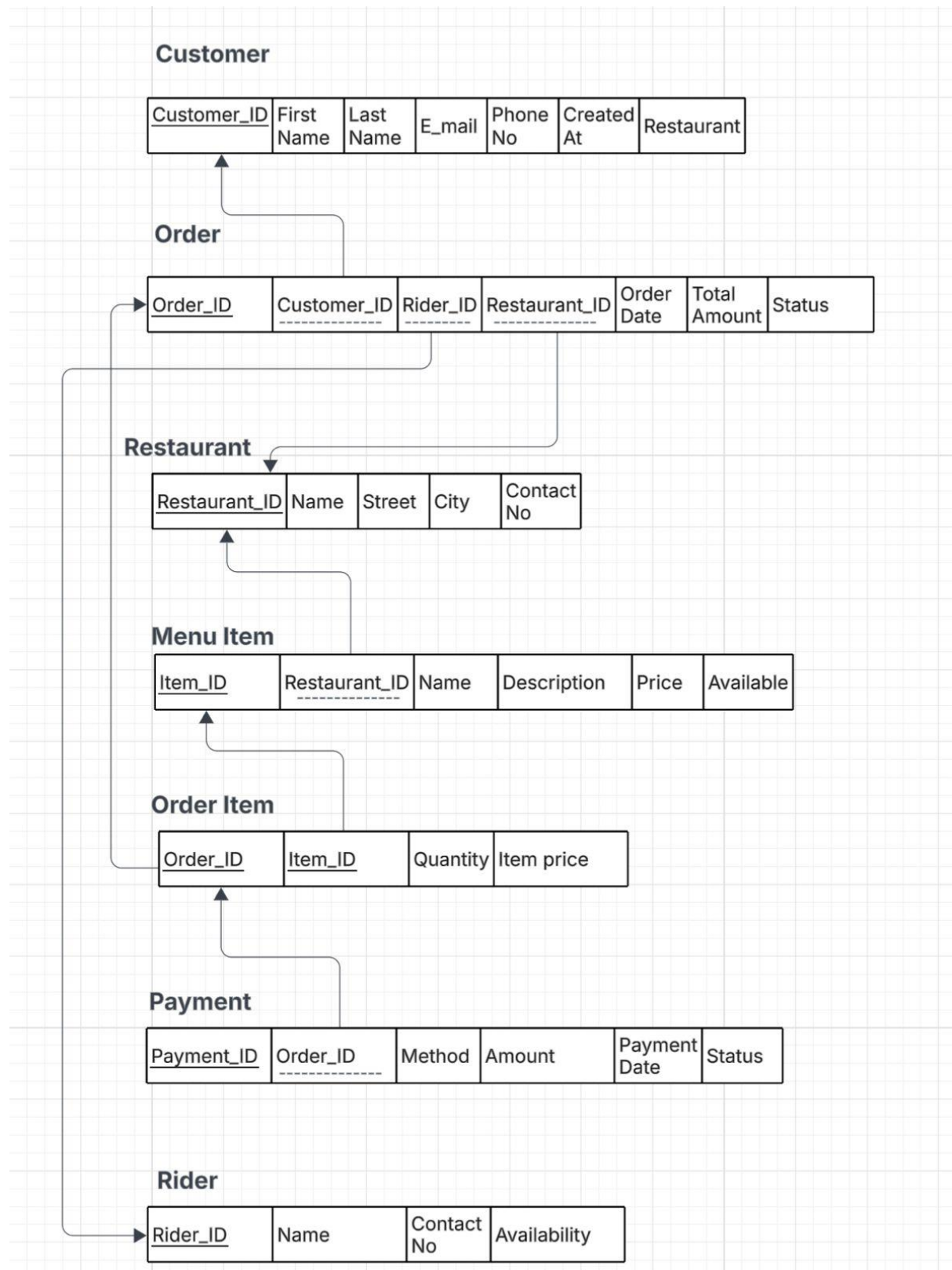
Rider.Contact No -> NVARCHAR(20)

- **Referential Integrity:**

Enforced by **FOREIGN KEY** (FKs reference existing PKs).

In our online food delivery system all Foreign key reference to existing Primary Key, shown in below final Relational Schema.

Final Relational Schema



Normalization:

1. First Normal Form (1NF)

Rule: All attributes must be atomic (single-valued), and there must be no repeating groups or multi-valued columns.

Customer

Attributes	Optional/ Required	Single / Composite	Simple/ Multi value	Stored/ Derived	Notes
Customer_ID	Required	Single	Simple	Stored	Primary Key
Name	Required	Composite (first/last)	Simple	Stored	
E_mail	Required	Single	Simple	Stored	Unique
Phone_NO	Required	Single	Multi value	Stored	Can be multiple
Created By	Required	Single	Simple	Stored	Date created
Address	Required	Single	Simple	Stored	

In above mention customer table Phone_No is multivalue we convert this into simple like just contain one value in it.

Customer

Attributes	Optional/ Required	Single / Composite	Simple/ Multi value	Stored/ Derived	Notes
Customer_ID	Required	Single	Simple	Stored	Primary Key
Name	Required	Composite (first/last)	Simple	Stored	
E_mail	Required	Single	Simple	Stored	Unique
Phone_NO	Required	Single	Simple	Stored	Cannot be multiple
Created By	Required	Single	Simple	Stored	Date created
Address	Required	Single	Simple	Stored	

Now this entity attributes are atomic there is no violation of 1NF.

Repeating Groups

The multi-valued attributes for line items are **resolved** by the **OrderItem** table (the associative entity). This eliminates the need for repeating columns like Item1_ID, Item2_ID, ... in the Order table.

CONCLUSION:All the other Entities and their attributes are atomic and the use of separate tables for entities and the associative entity (OrderItem) ensures all relations are in 1NF so the system is in 1NF form.

2. Second Normal Form (2NF)

Rule: The relation must be in 1NF, and no non-key attribute can depend only on a **part of a composite key** (no partial dependencies).

Single-Key Tables

All tables(Entities) with a single primary key:-

- Customer Table with Attributes
(**Customer_ID(PK)**,First Name,Last Name,E_mail,Phone_No,Created At)
- Restaurant Table with Attributes
(**Restaurant_ID(PK)**,Name,Street,City,Contact_No)
- Rider Table with Attributes
(**Rider_ID(PK)**,Name,Contact_No,Availability)
- Payment Table with Attributes
(**Payment_ID(PK)**,Order_ID(FK),Method,,Amount,Payment Date,Status)
- Order Table with Attributes
(**Order_ID(PK)**,Customer_ID(FK),Restaurant_ID(FK),Rider_ID(FK),Order Date, Total Amount,Status)
- MenuItem Table with Attributes
(**Item_ID(PK)**,Restaurant_ID(FK),Name,Description,Price,Available)

RESULT:Since the PK is a single attribute, there is no "part" of the key for non-key attributes to partially depend on.

Composite-Key Table.

- OrderItem Table with Attributes

(Order_ID(PK)/(FK),Item_ID(PK)/(FK),Quantity,Item Price)

OrderItem has the composite key (**OrderID + ItemID**). Its non-key attributes are **Quantity** and **Item Price**

RESULT: **Quantity** and **Item Price** depend on the **entire composite key**. The price recorded is the **historical price** for *that specific order instance* (OrderID), making it dependent on the full key for transactional integrity. If it depended only on **ItemID**, it would violate 2NF but it donot violate it and Quantity also depend on both part of primary key.

CONCLUSION: From above explanation it is prove that system is in 2NF Form.

3. Third Normal Form (3NF)

Rule: The relation must be in 2NF, and no non-key attribute can depend on another non-key attribute (no transitive dependencies).

All Entities with attributes:-

- Customer Table with Attributes

(**Customer_ID(PK)**,First Name,Last Name,E_mail,Phone_No,Created At)

Validation: All attributes depend only on **CustomerID**. No non-key attribute depends on another non-key attribute (e.g., **Phone No** does not determine **First Name**).

- Restaurant Table with Attributes

(**Restaurant_ID(PK)**,Name,Street,City,Contact_No)

Validation: All attributes depend only on **RestaurantID**. Descriptors are correctly placed here.

- Rider Table with Attributes

(**Rider_ID(PK)**,Name,Contact_No,Availability)

Validation: All attributes depend only on **RiderID**. This prevents descriptive rider data from being redundantly stored in the Order table.

- Payment Table with Attributes

(Payment_ID(PK),Order_ID(FK),Method,,Amount,Payment Date,Status)

Validation: All attributes depend solely on **Payment_ID**. The **Order_ID (FK)** links to the related order, but no other order details are stored here.

- Order Table with Attributes

(Order_ID(PK),Customer_ID(FK),Restaurant_ID(FK),Rider_ID(FK),Order Date, Total Amount,Status)

Validation: All Foreign Keys and descriptive attributes (Date, Total Amount, Status) depend only on the **OrderID**. Descriptive details about the Customer, Restaurant, or Rider are correctly isolated in their own tables.

- MenuItem Table with Attributes

(Item_ID(PK),Restaurant_ID(FK),Name,Description,Price,Available)

Validation: All non-key attributes depend solely on **ItemID**. The **RestaurantID (FK)** identifies the restaurant the item belongs to, but no other descriptive restaurant data is stored here.

- OrderItem Table with Attributes

(Order_ID(PK)/(FK),Item_ID(PK)/(FK),Quantity,Item Price)

Validation: Since this table already satisfied 2NF, and its non-key attributes are fully dependent on the composite key, there are no transitive dependencies present.

CONCLUSION:From above Explanation it is proved that our system is in 3NF.

Normalization Scenario

In this scenario we consider just one table and make it normalize. This table contains information about order, rider, menu item and order item for our system.

Order_Master_Details

Attributes	Description
Order_ID	Primary Key(PK)
Item_ID	Primary Key(PK)
Quantity	
Item Price	
Item name	
Item Description	
Customer_ID	Foreign Key(FK)
Restaurant_ID	Foreign Key(FK)
Rider_ID	Foreign Key(FK)
Order Date	
Total Amount	
Order Status	
Rider Name	
Contact NO	
Availability	

Normalizing to Second Normal Form (2NF)

Rule: Remove all partial dependencies.

The attributes *Item Name*, *Description*, and *Item Price* depend only on *ItemID*. We separate them to create the **MenuItem** table.

Now our final 2 tables will be:-

Order_Master_Details

Attributes	Description
Order_ID	Primary Key(PK)
Item_ID	Primary Key(PK)
Quantity	
Item Price	
Item name	
Item Description	
Customer_ID	Foreign Key(FK)

Restaurant_ID	Foreign Key(FK)
Rider_ID	Foreign Key(FK)
Order Date	
Total Amount	
Order Status	
Rider Name	
Contact NO	
Availability	

Menuitem

Attributes	Description
Item_ID	Primary Key(PK)
Item Price	
Item name	
Item Description	

This new table contain just detail relevant to menuitem in which **Item_ID** is PK.

The same **Item_ID** is FK in Order_Master_Details Table.

Normalizing to Third Normal Form (3NF)

Rule: Remove all transitive dependencies.

The attributes **Rider Name**, **Rider Contact No**, and **Availability** depend on the non-key attribute **RiderID**. We separate them to create the **Rider** table.

The remaining **Order_Details** table is then split into the main **Order** table and the associative **OrderItem** table.

Order_Master_Details

Attributes	Description
Order_ID	Primary Key(PK)
Item_ID	Primary Key(PK)
Quantity	
Item Price	
Item name	
Item Description	
Customer_ID	Foreign Key(FK)
Restaurant_ID	Foreign Key(FK)

Rider_ID	Foreign Key(FK)
Order Date	
Total Amount	
Order Status	
Rider Name	
Contact NO	
Availability	

Now our final 3 tables will be:-

Order

Attributes	Description
Order_ID	Primary Key(PK)
Customer_ID	Foreign Key(FK)
Restaurant_ID	Foreign Key(FK)
Rider_ID	Foreign Key(FK)
Order Date	
Total Amount	
Order Status	

OrderItem

Attributes	Description
Order_ID	Primary Key(PK)
Item_ID	Primary Key(PK)
Quantity	
Item Price	

Rider

Attributes	Description
Rider_ID	Primary Key(PK)
Rider Name	
Contact NO	
Availability	

CONCLUSION:Then we convert Order_Master_Details table into four (Menuitem,Order,Orderitem,Order) tables due to which all transitive and partial dependency is removed and our system is in 2NF and 3NF.

SQL Code

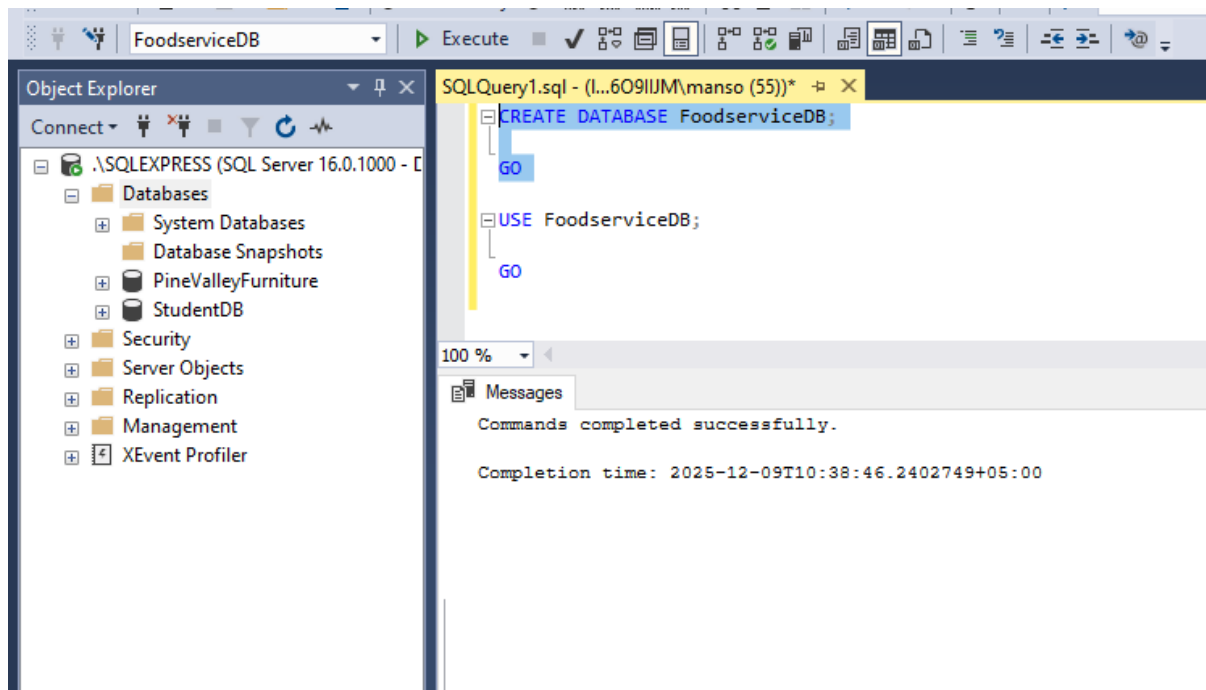
Database Setup

```
-- CREATE DATABASE FoodserviceDB;
```

```
--GO
```

```
USE FoodserviceDB;
```

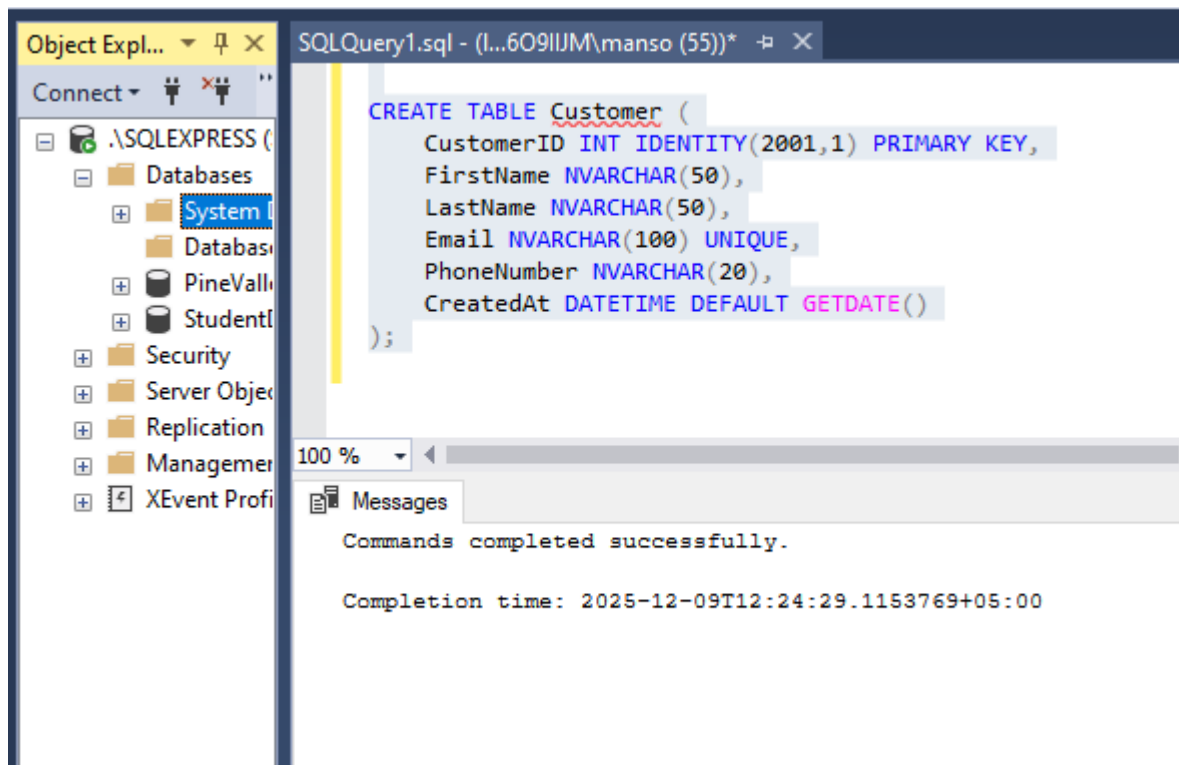
```
GO
```



TABLES

CUSTOMER TABLE

```
CREATE TABLE Customer (  
    CustomerID INT IDENTITY(2001,1) PRIMARY KEY,  
    FirstName NVARCHAR(50),  
    LastName NVARCHAR(50),  
    Email NVARCHAR(100) UNIQUE,  
    PhoneNumber NVARCHAR(20),  
    CreatedAt DATETIME DEFAULT GETDATE()  
);
```

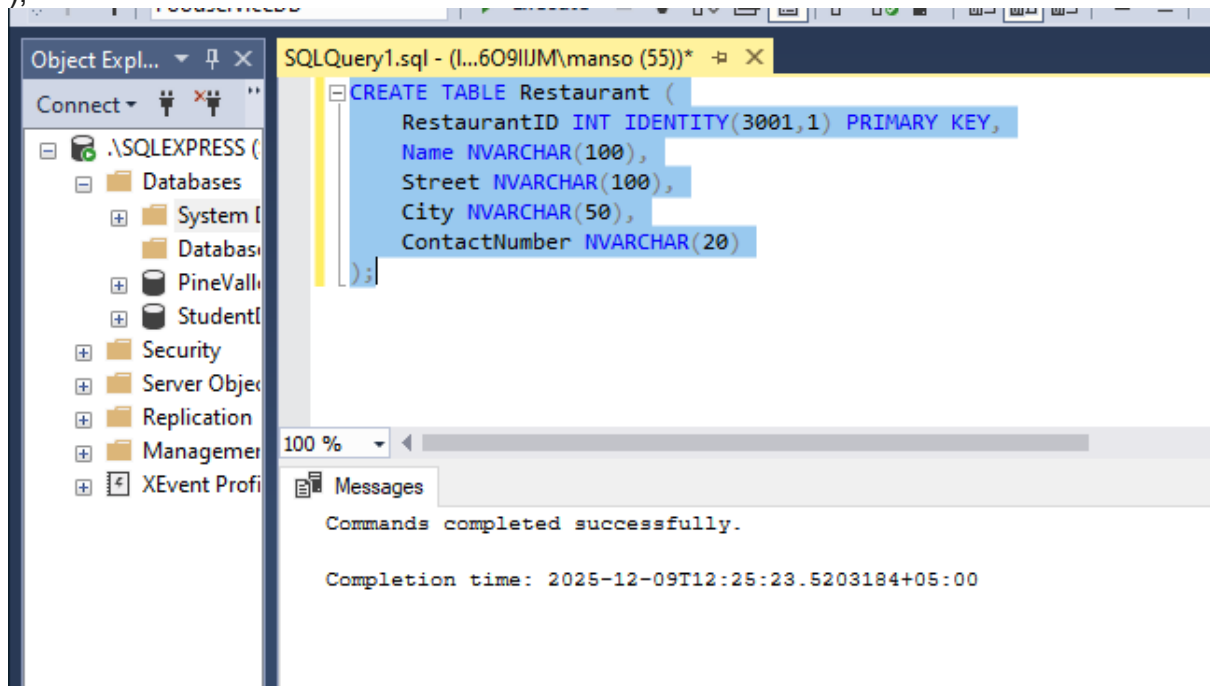


RESTAURANT TABLE

```

CREATE TABLE Restaurant (
    RestaurantID INT IDENTITY(3001,1) PRIMARY KEY,
    Name NVARCHAR(100),
    Street NVARCHAR(100),
    City NVARCHAR(50),
    ContactNumber NVARCHAR(20)
);

```



MENU ITEM TABLE

```
CREATE TABLE MenuItem (  
    ItemID INT IDENTITY(4001,1) PRIMARY KEY,  
    RestaurantID INT NOT NULL,  
    Name NVARCHAR(100),  
    Description NVARCHAR(255),  
    Price DECIMAL(10,2),  
    Available BIT,  
    CONSTRAINT FK_MenuItem_Restaurant FOREIGN KEY (RestaurantID)  
REFERENCES Restaurant(RestaurantID)  
);
```

RIDER TABLE

```
CREATE TABLE Rider (  
    RiderID INT IDENTITY(5001,1) PRIMARY KEY,  
    Name NVARCHAR(100),  
    ContactNumber NVARCHAR(20),  
    Availability BIT  
);
```

ORDER TABLE

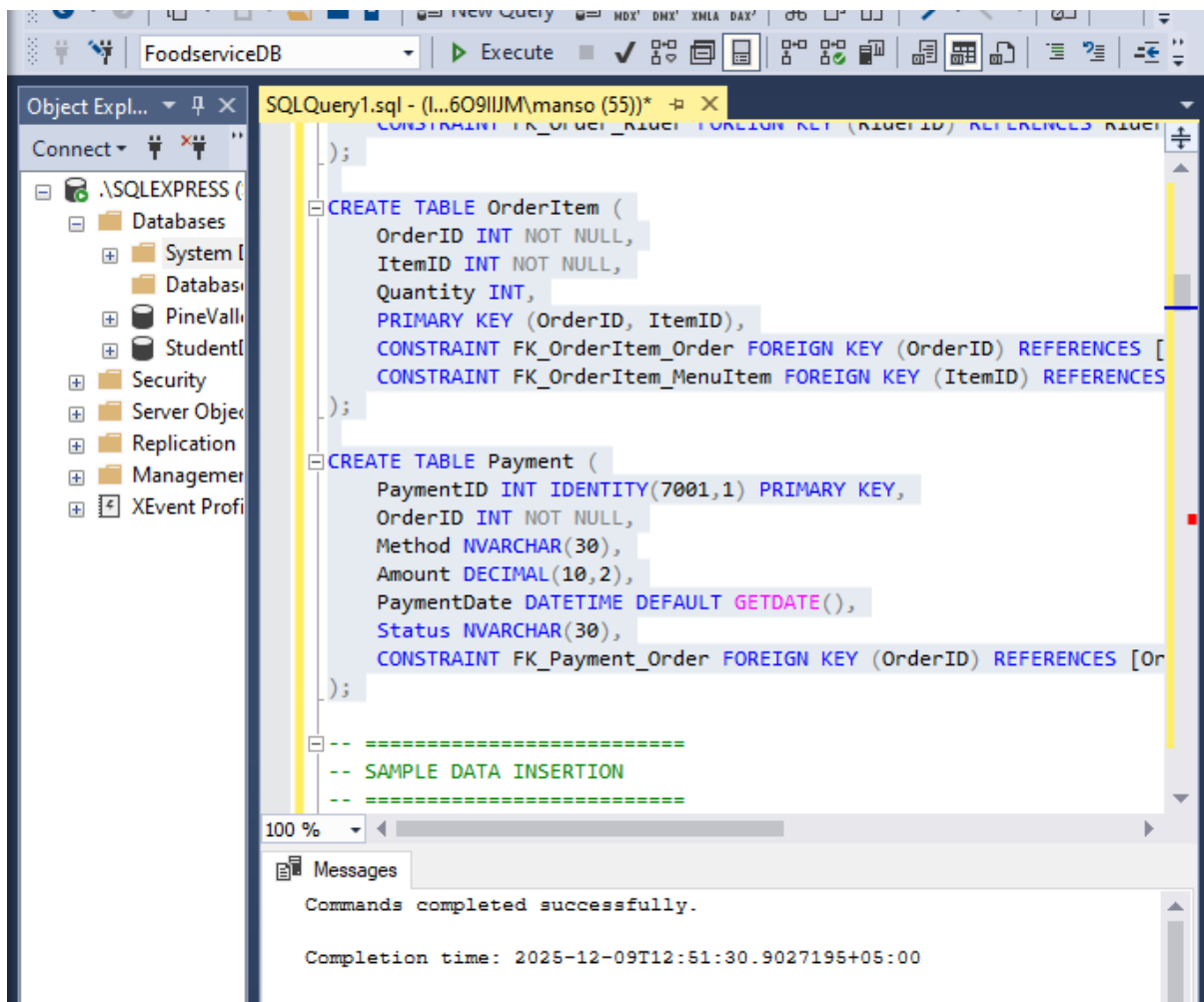
```
CREATE TABLE [Order] (  
    OrderID INT IDENTITY(6001,1) PRIMARY KEY,  
    CustomerID INT NOT NULL,  
    RestaurantID INT NOT NULL,  
    RiderID INT NOT NULL,  
    OrderDate DATETIME DEFAULT GETDATE(),  
    Status NVARCHAR(30),  
    CONSTRAINT FK_Order_Customer FOREIGN KEY (CustomerID)  
REFERENCES Customer(CustomerID),  
    CONSTRAINT FK_Order_Restaurant FOREIGN KEY (RestaurantID)  
REFERENCES Restaurant(RestaurantID),  
    CONSTRAINT FK_Order_Rider FOREIGN KEY (RiderID) REFERENCES  
Rider(RiderID)  
);
```

ORDER ITEM TABLE

```
CREATE TABLE OrderItem (  
    OrderID INT NOT NULL,  
    ItemID INT NOT NULL,  
    Quantity INT,  
    PRIMARY KEY (OrderID, ItemID),  
    CONSTRAINT FK_OrderItem_Order FOREIGN KEY (OrderID) REFERENCES  
[Order](OrderID),  
    CONSTRAINT FK_OrderItem_MenuItem FOREIGN KEY (ItemID) REFERENCES  
MenuItem(ItemID)  
);
```

PAYMENT TABLE

```
CREATE TABLE Payment (  
    PaymentID INT IDENTITY(7001,1) PRIMARY KEY,  
    OrderID INT NOT NULL,  
    Method NVARCHAR(30),  
    Amount DECIMAL(10,2),  
    PaymentDate DATETIME DEFAULT GETDATE(),  
    Status NVARCHAR(30),  
    CONSTRAINT FK_Payment_Order FOREIGN KEY (OrderID) REFERENCES  
[Order](OrderID)  
);
```



Data Insertion

Customers

```
INSERT INTO Customer (FirstName, LastName, Email, PhoneNumber)  
VALUES  
( 'Ali', 'Raza', 'ali1@example.com', '03001111111'),  
( 'Sara', 'Khan', 'sara2@example.com', '03002222222'),
```

```
( 'Hamza', 'Malik', 'hamza3@example.com', '03003333333'),
( 'Ayesha', 'Sheikh', 'ayesha4@example.com', '03004444444'),
( 'Bilal', 'Ahmed', 'bilal5@example.com', '03005555555'),
( 'Fatima', 'Javed', 'fatima6@example.com', '03006666666'),
( 'Usman', 'Butt', 'usman7@example.com', '03007777777'),
( 'Zain', 'Ali', 'zain8@example.com', '03008888888'),
( 'Hira', 'Saleem', 'hira9@example.com', '03009999999'),
( 'Taha', 'Rafay', 'taha10@example.com', '03110000001'),
( 'Rimsha', 'Shah', 'rimsha11@example.com', '03110000002'),
( 'Fahad', 'Khalid', 'fahad12@example.com', '03110000003'),
( 'Noor', 'Iqbal', 'noor13@example.com', '03110000004'),
( 'Danish', 'Hanif', 'danish14@example.com', '03110000005'),
( 'Sadaf', 'Zahid', 'sadaf15@example.com', '03110000006');
```

SQLQuery1.sql - (\\...609IUM\\manso (55))*

```
-- Customers
INSERT INTO Customer (FirstName, LastName, Email, PhoneNumber)
VALUES
( 'Ali', 'Raza', 'ali1@example.com', '03001111111'),
( 'Sara', 'Khan', 'sara2@example.com', '03002222222'),
( 'Hamza', 'Malik', 'hamza3@example.com', '03003333333'),
( 'Ayesha', 'Sheikh', 'ayesha4@example.com', '03004444444'),
( 'Bilal', 'Ahmed', 'bilal5@example.com', '03005555555'),
( 'Fatima', 'Javed', 'fatima6@example.com', '03006666666'),
```

	CustomerID	FirstName	LastName	Email	PhoneNumber	CreatedAt
1	2001	Ali	Raza	ali1@example.com	03001111111	2025-12-09 12:
2	2002	Sara	Khan	sara2@example.com	03002222222	2025-12-09 12:
3	2003	Hamza	Malik	hamza3@example.com	03003333333	2025-12-09 12:
4	2004	Ayesha	Sheikh	ayesha4@example.com	03004444444	2025-12-09 12:
5	2005	Bilal	Ahmed	bilal5@example.com	03005555555	2025-12-09 12:
6	2006	Fatima	Javed	fatima6@example.com	03006666666	2025-12-09 12:
7	2007	Usman	Butt	usman7@example.com	03007777777	2025-12-09 12:
8	2008	Zain	Ali	zain8@example.com	03008888888	2025-12-09 12:
9	2009	Hira	Saleem	hira9@example.com	03009999999	2025-12-09 12:
10	2010	Taha	Rafay	taha10@example.com	03110000001	2025-12-09 12:
11	2011	Rimsha	Shah	rimsha11@example.c...	03110000002	2025-12-09 12:
12	2012	Fahad	Khalid	fahad12@example.com	03110000003	2025-12-09 12:
13	2013	Noor	Iqbal	noor13@example.com	03110000004	2025-12-09 12:
14	2014	Danish	Hanif	danish14@example.c...	03110000005	2025-12-09 12:
15	2015	Sadaf	Zahid	sadaf15@example.com	03110000006	2025-12-09 12:

local)\SQLEXPRESS (16.0 RTM) | DESKTOP-609IUM\\manso ... | FoodserviceDB | 00:00:00 | 15 rows

Restaurants

```
INSERT INTO Restaurant (Name, Street, City, ContactNumber)
VALUES
```

('Pizza Palace','Main Road','Islamabad','0511111111'),
 ('Burger Hub','Blue Area','Islamabad','0512222222'),
 ('Biryani House','Saddar','Rawalpindi','0513333333'),
 ('Food Street Café','G-9','Islamabad','0514444444'),
 ('Tikka Junction','F-10','Islamabad','0515555555'),
 ('Karachi Bites','F-11','Islamabad','0516666666'),
 ('Rolling Pizza','Gulshan Road','Karachi','0211111111'),
 ('BBQ Tonight','Clifton','Karachi','0212222222'),
 ('Shawarma King','Johar Town','Lahore','0421111111'),
 ('Lahore Grill','Model Town','Lahore','0422222222'),
 ('Pasta Point','DHA Phase 2','Karachi','0213333333'),
 ('Kebab Villa','Bahria Town','Rawalpindi','0517777777'),
 ('Noodle Express','I-8','Islamabad','0518888888'),
 ('Zinger Stop','G-11','Islamabad','0519999999'),
 ('Fry Master','G-6','Islamabad','0511212121');

Object Explorer | SQLQuery1.sql - (\\...609IUM\manso (55))*

```

-- Customers
INSERT INTO Customer (FirstName, LastName, Email, PhoneNumber)
VALUES
('Ali', 'Raza', 'ali1@example.com', '0300111111'),
('Sara', 'Khan', 'sara2@example.com', '0300222222'),
('Hamza', 'Malik', 'hamza3@example.com', '0300333333'),
('Ayesha', 'Sheikh', 'ayesha4@example.com', '0300444444'),
('Bilal', 'Ahmed', 'bilal5@example.com', '0300555555'),
('Fatima', 'Javed', 'fatima6@example.com', '0300666666').
  
```

100 %

Results | Messages

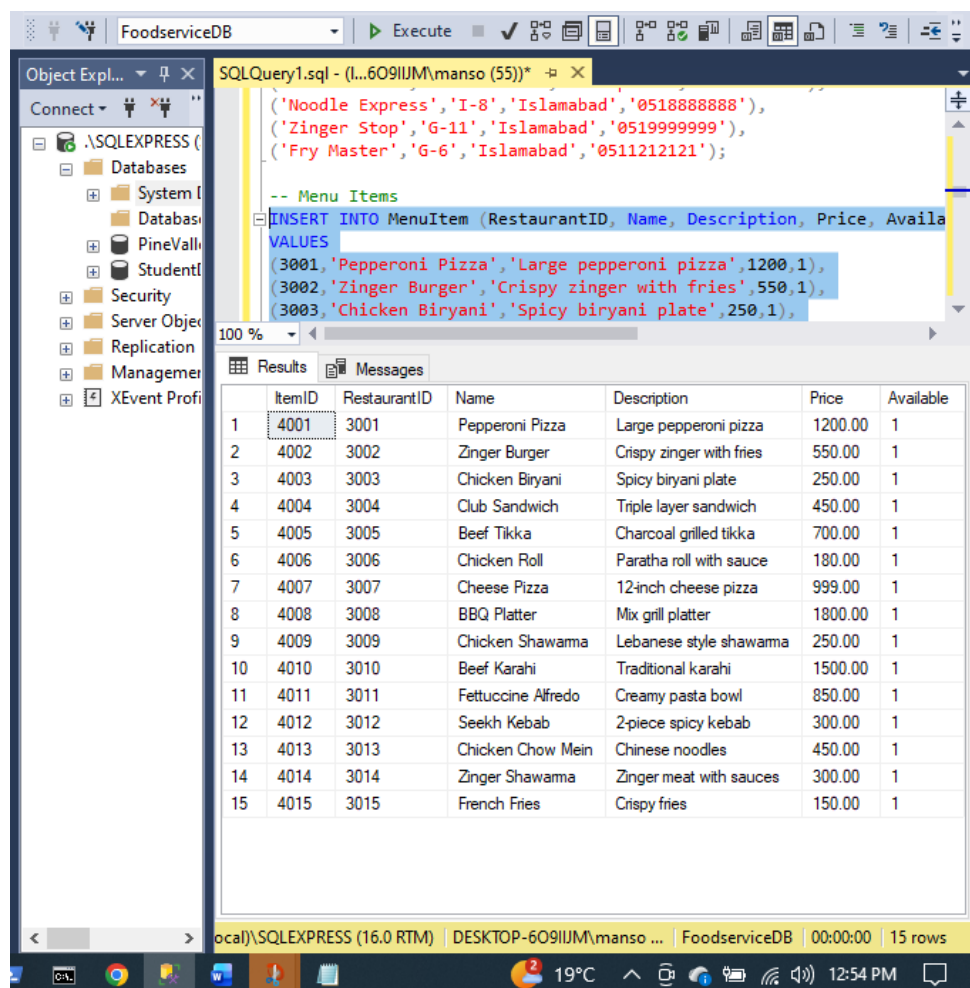
	CustomerID	FirstName	LastName	Email	PhoneNumber	CreatedAt
1	2001	Ali	Raza	ali1@example.com	0300111111	2025-12-09 12:
2	2002	Sara	Khan	sara2@example.com	0300222222	2025-12-09 12:
3	2003	Hamza	Malik	hamza3@example.com	0300333333	2025-12-09 12:
4	2004	Ayesha	Sheikh	ayesha4@example.com	0300444444	2025-12-09 12:
5	2005	Bilal	Ahmed	bilal5@example.com	0300555555	2025-12-09 12:
6	2006	Fatima	Javed	fatima6@example.com	0300666666	2025-12-09 12:
7	2007	Usman	Butt	usman7@example.com	0300777777	2025-12-09 12:
8	2008	Zain	Ali	zain8@example.com	0300888888	2025-12-09 12:
9	2009	Hira	Saleem	hira9@example.com	0300999999	2025-12-09 12:
10	2010	Taha	Rafay	taha10@example.com	0311000001	2025-12-09 12:
11	2011	Rimsha	Shah	rimsha11@example.c...	0311000002	2025-12-09 12:
12	2012	Fahad	Khalid	fahad12@example.com	0311000003	2025-12-09 12:
13	2013	Noor	Iqbal	noor13@example.com	0311000004	2025-12-09 12:
14	2014	Danish	Hanif	danish14@example.c...	0311000005	2025-12-09 12:
15	2015	Sadaf	Zahid	sadaf15@example.com	0311000006	2025-12-09 12:

local)\SQLEXPRESS (16.0 RTM) | DESKTOP-609IUM\manso ... | FoodserviceDB | 00:00:00 | 15 rows

Menu Items

INSERT INTO MenuItem (RestaurantID, Name, Description, Price, Available)
VALUES

```
(3001,'Pepperoni Pizza','Large pepperoni pizza',1200,1),  
(3002,'Zinger Burger','Crispy zinger with fries',550,1),  
(3003,'Chicken Biryani','Spicy biryani plate',250,1),  
(3004,'Club Sandwich','Triple layer sandwich',450,1),  
(3005,'Beef Tikka','Charcoal grilled tikka',700,1),  
(3006,'Chicken Roll','Paratha roll with sauce',180,1),  
(3007,'Cheese Pizza','12-inch cheese pizza',999,1),  
(3008,'BBQ Platter','Mix grill platter',1800,1),  
(3009,'Chicken Shawarma','Lebanese style shawarma',250,1),  
(3010,'Beef Karahi','Traditional karahi',1500,1),  
(3011,'Fettuccine Alfredo','Creamy pasta bowl',850,1),  
(3012,'Seekh Kebab','2-piece spicy kebab',300,1),  
(3013,'Chicken Chow Mein','Chinese noodles',450,1),  
(3014,'Zinger Shawarma','Zinger meat with sauces',300,1),  
(3015,'French Fries','Crispy fries',150,1);
```



The screenshot shows a SQL Server Enterprise Manager interface. The SQL Query window displays the following query:

```
-- Menu Items  
INSERT INTO MenuItem (RestaurantID, Name, Description, Price, Available)  
VALUES  
(3001, 'Pepperoni Pizza', 'Large pepperoni pizza', 1200, 1),  
(3002, 'Zinger Burger', 'Crispy zinger with fries', 550, 1),  
(3003, 'Chicken Biryani', 'Spicy biryani plate', 250, 1),  
(3004, 'Club Sandwich', 'Triple layer sandwich', 450, 1),  
(3005, 'Beef Tikka', 'Charcoal grilled tikka', 700, 1),  
(3006, 'Chicken Roll', 'Paratha roll with sauce', 180, 1),  
(3007, 'Cheese Pizza', '12-inch cheese pizza', 999, 1),  
(3008, 'BBQ Platter', 'Mix grill platter', 1800, 1),  
(3009, 'Chicken Shawarma', 'Lebanese style shawarma', 250, 1),  
(3010, 'Beef Karahi', 'Traditional karahi', 1500, 1),  
(3011, 'Fettuccine Alfredo', 'Creamy pasta bowl', 850, 1),  
(3012, 'Seekh Kebab', '2-piece spicy kebab', 300, 1),  
(3013, 'Chicken Chow Mein', 'Chinese noodles', 450, 1),  
(3014, 'Zinger Shawarma', 'Zinger meat with sauces', 300, 1),  
(3015, 'French Fries', 'Crispy fries', 150, 1);
```

The Results pane shows the following data:

ItemID	RestaurantID	Name	Description	Price	Available
4001	3001	Pepperoni Pizza	Large pepperoni pizza	1200.00	1
4002	3002	Zinger Burger	Crispy zinger with fries	550.00	1
4003	3003	Chicken Biryani	Spicy biryani plate	250.00	1
4004	3004	Club Sandwich	Triple layer sandwich	450.00	1
4005	3005	Beef Tikka	Charcoal grilled tikka	700.00	1
4006	3006	Chicken Roll	Paratha roll with sauce	180.00	1
4007	3007	Cheese Pizza	12-inch cheese pizza	999.00	1
4008	3008	BBQ Platter	Mix grill platter	1800.00	1
4009	3009	Chicken Shawarma	Lebanese style shawarma	250.00	1
4010	3010	Beef Karahi	Traditional karahi	1500.00	1
4011	3011	Fettuccine Alfredo	Creamy pasta bowl	850.00	1
4012	3012	Seekh Kebab	2-piece spicy kebab	300.00	1
4013	3013	Chicken Chow Mein	Chinese noodles	450.00	1
4014	3014	Zinger Shawarma	Zinger meat with sauces	300.00	1
4015	3015	French Fries	Crispy fries	150.00	1

Riders

INSERT INTO Rider (Name, ContactNumber, Availability)
VALUES

```
('Ahmed','03450000001',1),
```

```

('Bilal','03450000002',1),
('Usman','03450000003',1),
('Hamza','03450000004',1),
('Sajid','03450000005',1),
('Jawad','03450000006',1),
('Tariq','03450000007',1),
('Ali Shan','03450000008',1),
('Noman','03450000009',1),
('Hussain','03450000010',1),
('Adeel','03450000011',1),
('Kashif','03450000012',1),
('Raza','03450000013',1),
('Daniyal','03450000014',1),
('Hammad','03450000015',1);

```

The screenshot shows the SQL Server Enterprise Manager interface. The query window displays the following SQL statement:

```

INSERT INTO Rider (Name, ContactNumber, Availability)
VALUES
('Ahmed', '03450000001', 1),
('Bilal', '03450000002', 1),
('Usman', '03450000003', 1),
('Hamza', '03450000004', 1),
('Sajid', '03450000005', 1),
('Jawad', '03450000006', 1),
('Tariq', '03450000007', 1),
('Ali Shan', '03450000008', 1),
('Noman', '03450000009', 1),
('Hussain', '03450000010', 1),
('Adeel', '03450000011', 1),
('Kashif', '03450000012', 1),
('Raza', '03450000013', 1),
('Daniyal', '03450000014', 1),
('Hammad', '03450000015', 1);

```

The Results pane shows the following data:

RiderID	Name	ContactNumber	Availability
1	Ahmed	03450000001	1
2	Bilal	03450000002	1
3	Usman	03450000003	1
4	Hamza	03450000004	1
5	Sajid	03450000005	1
6	Jawad	03450000006	1
7	Tariq	03450000007	1
8	Ali Shan	03450000008	1
9	Noman	03450000009	1
10	Hussain	03450000010	1
11	Adeel	03450000011	1
12	Kashif	03450000012	1
13	Raza	03450000013	1
14	Daniyal	03450000014	1
15	Hammad	03450000015	1

The status bar at the bottom indicates: local)\SQLEXPRESS (16.0 RTM) | DESKTOP-609IJJM\manso ... | FoodserviceDB | 00:00:00 | 15 rows

orders

INSERT INTO [Order] (CustomerID, RestaurantID, RiderID, Status)

VALUES

```
(2001,3001,5001,'Delivered'),
(2002,3002,5002,'Pending'),
(2003,3003,5003,'Preparing'),
(2004,3004,5004,'Delivered'),
(2005,3005,5005,'Pending'),
(2006,3006,5006,'Preparing'),
(2007,3007,5007,'Delivered'),
(2008,3008,5008,'Pending'),
(2009,3009,5009,'Preparing'),
(2010,3010,5010,'Delivered'),
(2011,3011,5011,'Pending'),
(2012,3012,5012,'Preparing'),
(2013,3013,5013,'Delivered'),
(2014,3014,5014,'Pending'),
(2015,3015,5015,'Delivered');
```

SQLQuery1.sql - (L...609IUM\manso (55))*

```
( 'Hammad', '03450000015', 1);
Select * from Rider;
-- Orders
INSERT INTO [Order] (CustomerID, RestaurantID, RiderID, Status)
VALUES
(2001,3001,5001,'Delivered'),
(2002,3002,5002,'Pending'),
(2003,3003,5003,'Preparing'),
(2004,3004,5004,'Delivered'),
(2005,3005,5005,'Pending'),
```

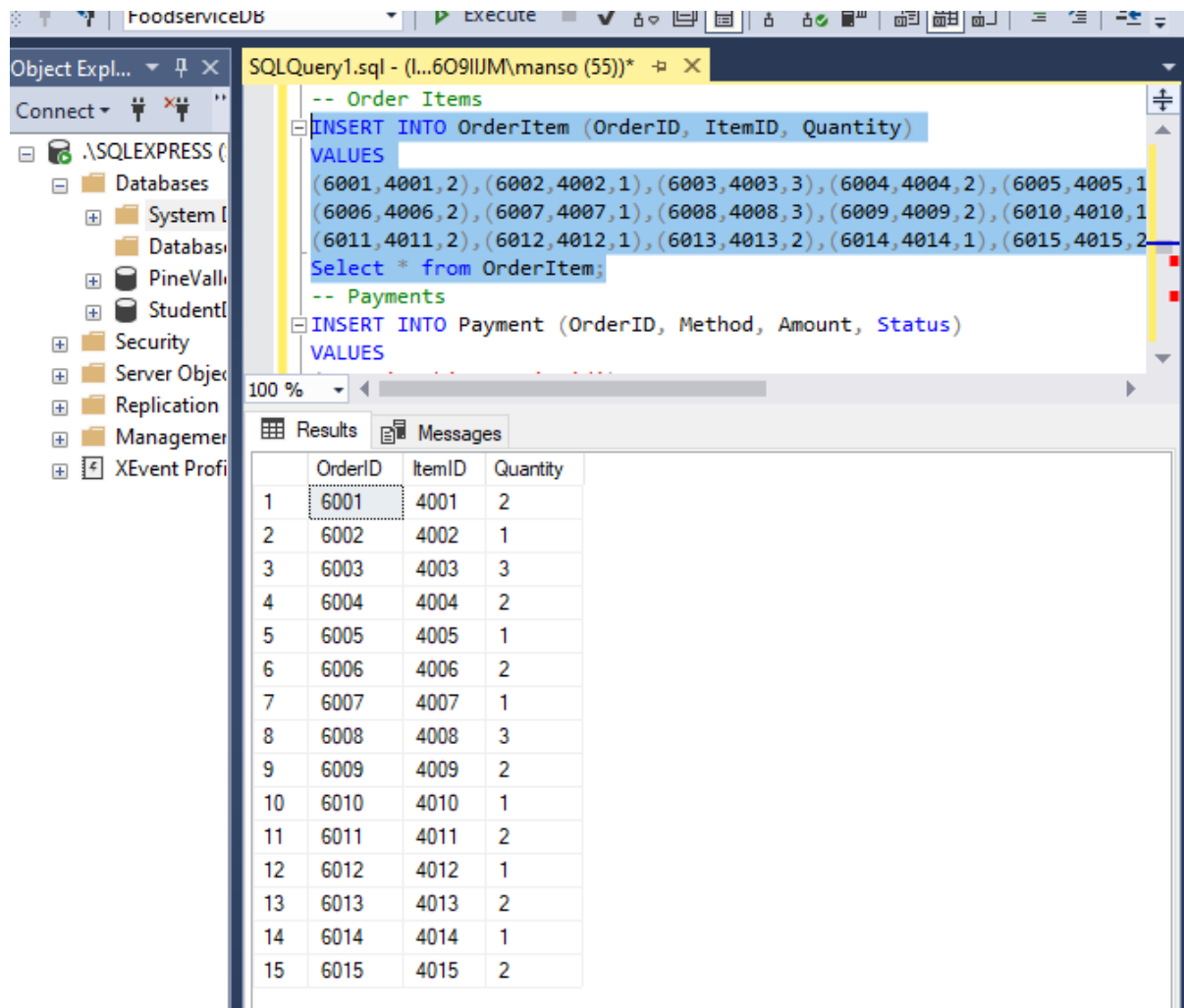
	OrderID	CustomerID	RestaurantID	RiderID	OrderDate	Status
1	6001	2001	3001	5001	2025-12-09 12:56:51.897	Delivered
2	6002	2002	3002	5002	2025-12-09 12:56:51.897	Pending
3	6003	2003	3003	5003	2025-12-09 12:56:51.897	Preparing
4	6004	2004	3004	5004	2025-12-09 12:56:51.897	Delivered
5	6005	2005	3005	5005	2025-12-09 12:56:51.897	Pending
6	6006	2006	3006	5006	2025-12-09 12:56:51.897	Preparing
7	6007	2007	3007	5007	2025-12-09 12:56:51.897	Delivered
8	6008	2008	3008	5008	2025-12-09 12:56:51.897	Pending
9	6009	2009	3009	5009	2025-12-09 12:56:51.897	Preparing
10	6010	2010	3010	5010	2025-12-09 12:56:51.897	Delivered
11	6011	2011	3011	5011	2025-12-09 12:56:51.897	Pending
12	6012	2012	3012	5012	2025-12-09 12:56:51.897	Preparing
13	6013	2013	3013	5013	2025-12-09 12:56:51.897	Delivered
14	6014	2014	3014	5014	2025-12-09 12:56:51.897	Pending
15	6015	2015	3015	5015	2025-12-09 12:56:51.897	Delivered

local)\SQLEXPRESS (16.0 RTM) | DESKTOP-609IUM\manso ... | FoodserviceDB | 00:00:00 | 15 rows

OrderItem

```
INSERT INTO OrderItem (OrderID, ItemID, Quantity)
VALUES
```

```
(6001,4001,2),(6002,4002,1),(6003,4003,3),(6004,4004,2),(6005,4005,1),
(6006,4006,2),(6007,4007,1),(6008,4008,3),(6009,4009,2),(6010,4010,1),
(6011,4011,2),(6012,4012,1),(6013,4013,2),(6014,4014,1),(6015,4015,2);
```



The screenshot shows a SQL Server Enterprise Manager window with a query execution window open. The query window displays the following SQL code:

```
-- Order Items
INSERT INTO OrderItem (OrderID, ItemID, Quantity)
VALUES
(6001,4001,2), (6002,4002,1), (6003,4003,3), (6004,4004,2), (6005,4005,1),
(6006,4006,2), (6007,4007,1), (6008,4008,3), (6009,4009,2), (6010,4010,1),
(6011,4011,2), (6012,4012,1), (6013,4013,2), (6014,4014,1), (6015,4015,2);
Select * from OrderItem;

-- Payments
INSERT INTO Payment (OrderID, Method, Amount, Status)
VALUES
```

The Results tab shows the following data:

	OrderID	ItemID	Quantity
1	6001	4001	2
2	6002	4002	1
3	6003	4003	3
4	6004	4004	2
5	6005	4005	1
6	6006	4006	2
7	6007	4007	1
8	6008	4008	3
9	6009	4009	2
10	6010	4010	1
11	6011	4011	2
12	6012	4012	1
13	6013	4013	2
14	6014	4014	1
15	6015	4015	2

Payments

```
INSERT INTO Payment (OrderID, Method, Amount, Status)
VALUES
```

```
(6001,'Cash',2400,'Paid'),
(6002,'Credit Card',550,'Pending'),
(6003,'Cash',750,'Paid'),
(6004,'Easypaisa',450,'Paid'),
(6005,'Cash',700,'Pending'),
(6006,'JazzCash',360,'Paid'),
(6007,'Credit Card',999,'Paid'),
(6008,'Cash',1800,'Pending'),
(6009,'Cash',250,'Paid'),
(6010,'Credit Card',1500,'Pending'),
(6011,'Cash',850,'Paid'),
(6012,'Credit Card',300,'Pending'),
```

(6013,'JazzCash',450,'Paid'),
 (6014,'Cash',300,'Paid'),
 (6015,'Easypaisa',150,'Pending');

The screenshot shows the Microsoft SQL Server Enterprise Manager interface. The 'Object Explorer' on the left shows the 'FoodserviceDB' database. The 'SQLQuery1.sql' window displays an INSERT statement into the 'Payment' table. The 'Results' pane shows 15 rows of data.

SQL Query:

```
INSERT INTO Payment (OrderID, Method, Amount, Status)
VALUES
(6001, 'Cash', 2400, 'Paid'),
(6002, 'Credit Card', 550, 'Pending'),
(6003, 'Cash', 750, 'Paid'),
(6004, 'Easypaisa', 450, 'Paid'),
(6005, 'Cash', 700, 'Pending'),
(6006, 'JazzCash', 360, 'Paid'),
(6007, 'Credit Card', 999, 'Paid'),
(6008, 'Cash', 1800, 'Pending'),
(6009, 'Cash', 250, 'Paid'),
(6010, 'Credit Card', 1500, 'Pending'),
(6011, 'Cash', 850, 'Paid'),
(6012, 'Credit Card', 300, 'Pending'),
(6013, 'JazzCash', 450, 'Paid'),
(6014, 'Cash', 300, 'Paid'),
(6015, 'Easypaisa', 150, 'Pending');
```

Results Table:

PaymentID	OrderID	Method	Amount	PaymentDate	Status	
1	7001	6001	Cash	2400.00	2025-12-09 12:58:19.830	Paid
2	7002	6002	Credit Card	550.00	2025-12-09 12:58:19.830	Pending
3	7003	6003	Cash	750.00	2025-12-09 12:58:19.830	Paid
4	7004	6004	Easypaisa	450.00	2025-12-09 12:58:19.830	Paid
5	7005	6005	Cash	700.00	2025-12-09 12:58:19.830	Pending
6	7006	6006	JazzCash	360.00	2025-12-09 12:58:19.830	Paid
7	7007	6007	Credit Card	999.00	2025-12-09 12:58:19.830	Paid
8	7008	6008	Cash	1800.00	2025-12-09 12:58:19.830	Pending
9	7009	6009	Cash	250.00	2025-12-09 12:58:19.830	Paid
10	7010	6010	Credit Card	1500.00	2025-12-09 12:58:19.830	Pending
11	7011	6011	Cash	850.00	2025-12-09 12:58:19.830	Paid
12	7012	6012	Credit Card	300.00	2025-12-09 12:58:19.830	Pending
13	7013	6013	JazzCash	450.00	2025-12-09 12:58:19.830	Paid
14	7014	6014	Cash	300.00	2025-12-09 12:58:19.830	Paid
15	7015	6015	Easypaisa	150.00	2025-12-09 12:58:19.830	Pending

Table Name	ID Column	Starts From
Customer	CustomerID	2001
Restaurant	RestaurantID	3001
MenuItem	ItemID	4001
Rider	RiderID	5001
Order	OrderID	6001
OrderItem	(composite PK)	Follows OrderID + ItemID FK references
Payment	PaymentID	7001

Users, Roles, and Access Control (DCL)

```
USE master;
```

```
GO
```

-- Create SQL Server Logins (for app login)

-- Each user type can log in through its own account

```
CREATE LOGIN CustomerLogin WITH PASSWORD = 'Customer@123';
```

```
CREATE LOGIN RiderLogin WITH PASSWORD = 'Rider@123';
```

```
CREATE LOGIN RestaurantLogin WITH PASSWORD = 'Restaurant@123';
```

```
CREATE LOGIN AdminLogin WITH PASSWORD = 'Admin@123';
```

```
GO
```

-- Create database users inside FoodserviceDB

```
USE FoodserviceDB;
```

```
GO
```

```
CREATE USER CustomerUser FOR LOGIN CustomerLogin;
```

```
CREATE USER RiderUser FOR LOGIN RiderLogin;
```

```
CREATE USER RestaurantUser FOR LOGIN RestaurantLogin;
```

```
CREATE USER AdminUser FOR LOGIN AdminLogin;
```

```
GO
```

--Create Roles

```
CREATE ROLE Customer_Role;
```

```
CREATE ROLE Rider_Role;
```

```
CREATE ROLE Restaurant_Manager_Role;
```

```
CREATE ROLE Admin_Role;  
GO
```

--Assign Permissions to Roles

-- Admin Role: Full access

```
GRANT SELECT, INSERT, UPDATE, DELETE ON Customer TO  
Admin_Role;  
  
GRANT SELECT, INSERT, UPDATE, DELETE ON Restaurant TO  
Admin_Role;  
  
GRANT SELECT, INSERT, UPDATE, DELETE ON MenuItem TO  
Admin_Role;  
  
GRANT SELECT, INSERT, UPDATE, DELETE ON Rider TO Admin_Role;  
  
GRANT SELECT, INSERT, UPDATE, DELETE ON [Order] TO  
Admin_Role;  
  
GRANT SELECT, INSERT, UPDATE, DELETE ON OrderItem TO  
Admin_Role;  
  
GRANT SELECT, INSERT, UPDATE, DELETE ON Payment TO  
Admin_Role;
```

-- Restaurant Manager Role: Manage menu & restaurant info

```
GRANT SELECT, INSERT, UPDATE ON MenuItem TO  
Restaurant_Manager_Role;  
  
GRANT SELECT, UPDATE ON Restaurant TO  
Restaurant_Manager_Role;
```

-- Rider Role: Can view orders & update own availability

```
GRANT SELECT ON [Order] TO Rider_Role;  
  
GRANT UPDATE ON Rider TO Rider_Role;
```

-- Customer Role: Read-only access to restaurants, menu, and own orders

```
GRANT SELECT ON Restaurant TO Customer_Role;
```

```
GRANT SELECT ON MenuItem TO Customer_Role;
```

```
GRANT SELECT ON [Order] TO Customer_Role;
```

```
GO
```

--Assign Users to Roles

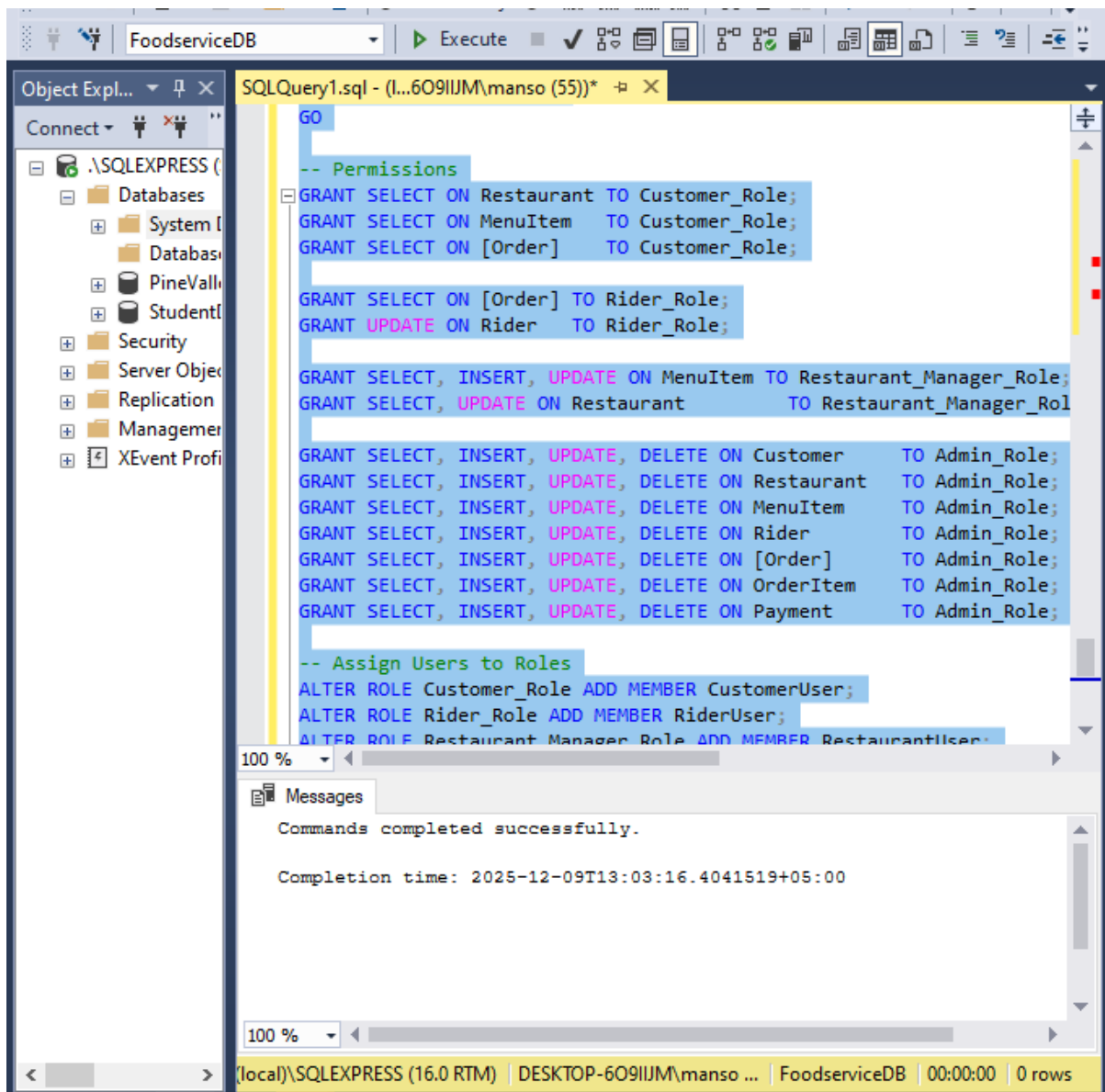
```
ALTER ROLE Customer_Role ADD MEMBER CustomerUser;
```

```
ALTER ROLE Rider_Role ADD MEMBER RiderUser;
```

```
ALTER ROLE Restaurant_Manager_Role ADD MEMBER RestaurantUser;
```

```
ALTER ROLE Admin_Role ADD MEMBER AdminUser;
```

```
GO
```

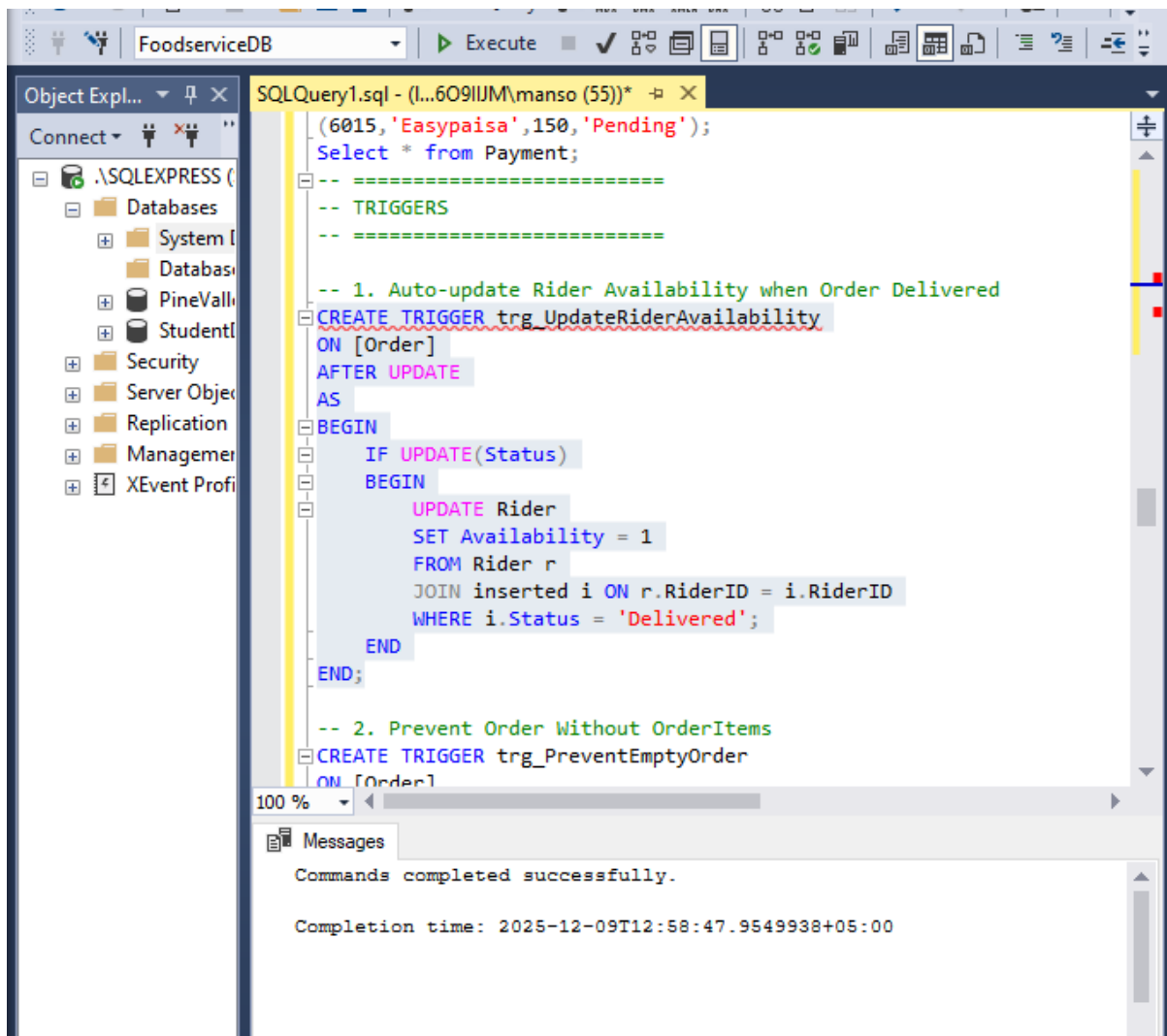


TRIGGERS

1.Auto-update Rider Availability when Order Delivered

- Automatically updates the assigned **Rider's Availability to 1** whenever the associated order's Status is changed to 'Delivered'.
- Enforces the business logic that a rider is immediately available for a new dispatch once their current delivery is completed.

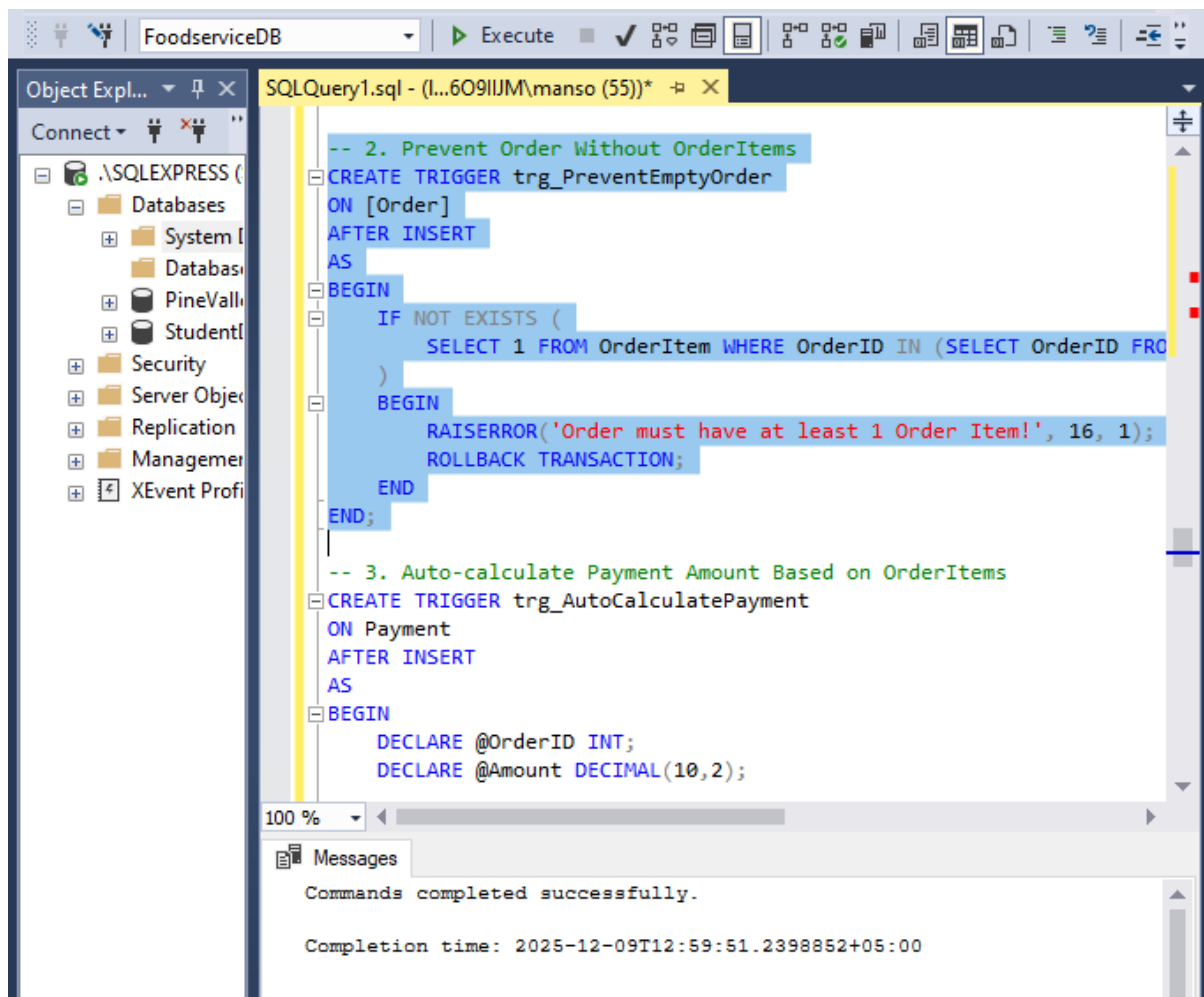
```
CREATE TRIGGER trg_UpdateRiderAvailability
ON [Order]
AFTER UPDATE
AS
BEGIN
    IF UPDATE(Status)
    BEGIN
        UPDATE Rider
        SET Availability = 1
        FROM Rider r
        JOIN inserted i ON r.RiderID = i.RiderID
        WHERE i.Status = 'Delivered';
    END
END;
```



2.Prevent Order Without OrderItems

- An AFTER INSERT validation check that ensures a newly created **Order** record has at least one corresponding entry in the **OrderItem** table.
- Prevents the commitment of invalid or meaningless orders by rolling back the entire transaction if the order has no line items.

```
CREATE TRIGGER trg_PreventEmptyOrder
ON [Order]
AFTER INSERT
AS
BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM OrderItem WHERE OrderID IN (SELECT OrderID FROM
inserted)
    )
    BEGIN
        RAISERROR('Order must have at least 1 Order Item!', 16, 1);
        ROLLBACK TRANSACTION;
    END
END;
```



3.Auto-calculate Payment Amount Based on OrderItems

- Automatically calculates the final **Amount** for a new **Payment** record by summing the total cost (Quantity * ItemPrice) from all related **OrderItem** records.
- Guarantees financial consistency and prevents manual data entry errors by ensuring the payment amount always matches the exact order total.

```
CREATE TRIGGER trg_AutoCalculatePayment
```

```
ON Payment
```

```
AFTER INSERT
```

```
AS
```

```
BEGIN
```

```
    DECLARE @OrderID INT;
```

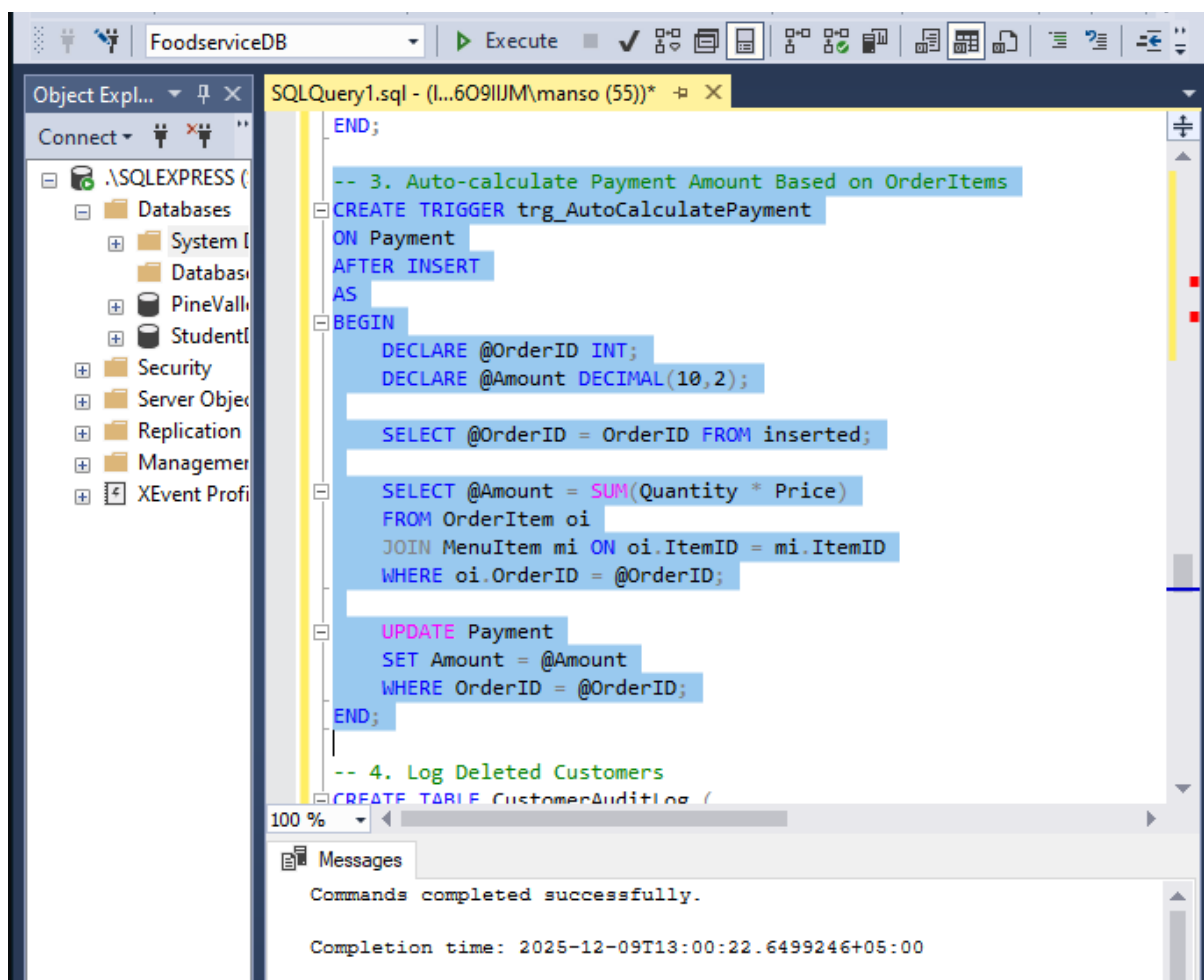
```
    DECLARE @Amount DECIMAL(10,2);
```

```
SELECT @OrderID = OrderID FROM inserted;
```

```
SELECT @Amount = SUM(Quantity * Price)
FROM OrderItem oi
JOIN MenuItem mi ON oi.ItemID = mi.ItemID
WHERE oi.OrderID = @OrderID;
```

```
UPDATE Payment
SET Amount = @Amount
WHERE OrderID = @OrderID;
```

```
END;
```

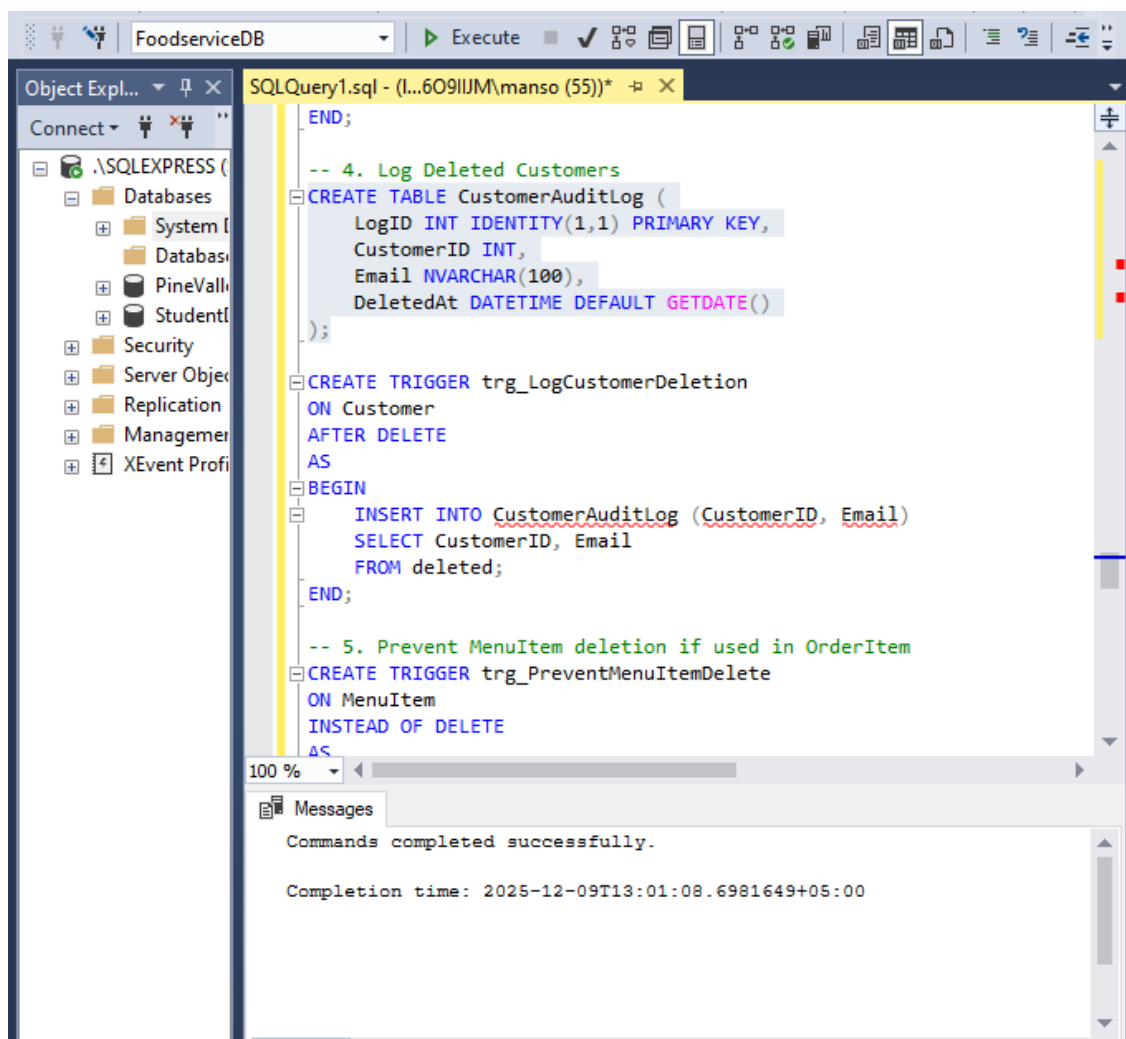


4. Log Deleted Customers

- A crucial auditing mechanism that fires AFTER DELETE on the **Customer** table, capturing the deleted record's details (CustomerID, Email).
- Maintains a mandatory, non-repudiable **Audit Log** of all removed customer accounts for historical tracking and compliance.

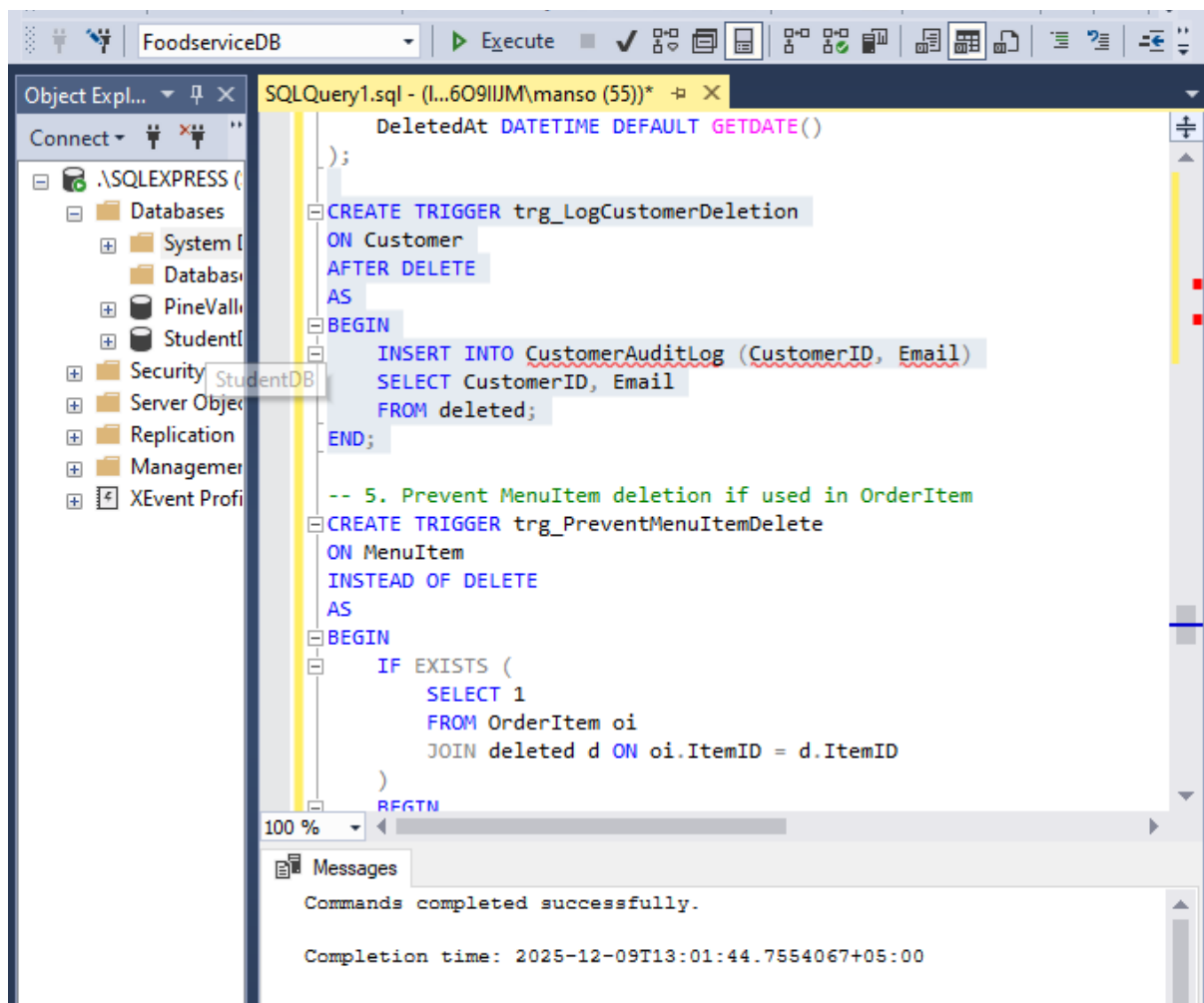
Create audit table:

```
CREATE TABLE CustomerAuditLog (  
    LogID INT IDENTITY(1,1) PRIMARY KEY,  
    CustomerID INT,  
    Email NVARCHAR(100),  
    DeletedAt DATETIME DEFAULT GETDATE()  
);
```



Trigger:

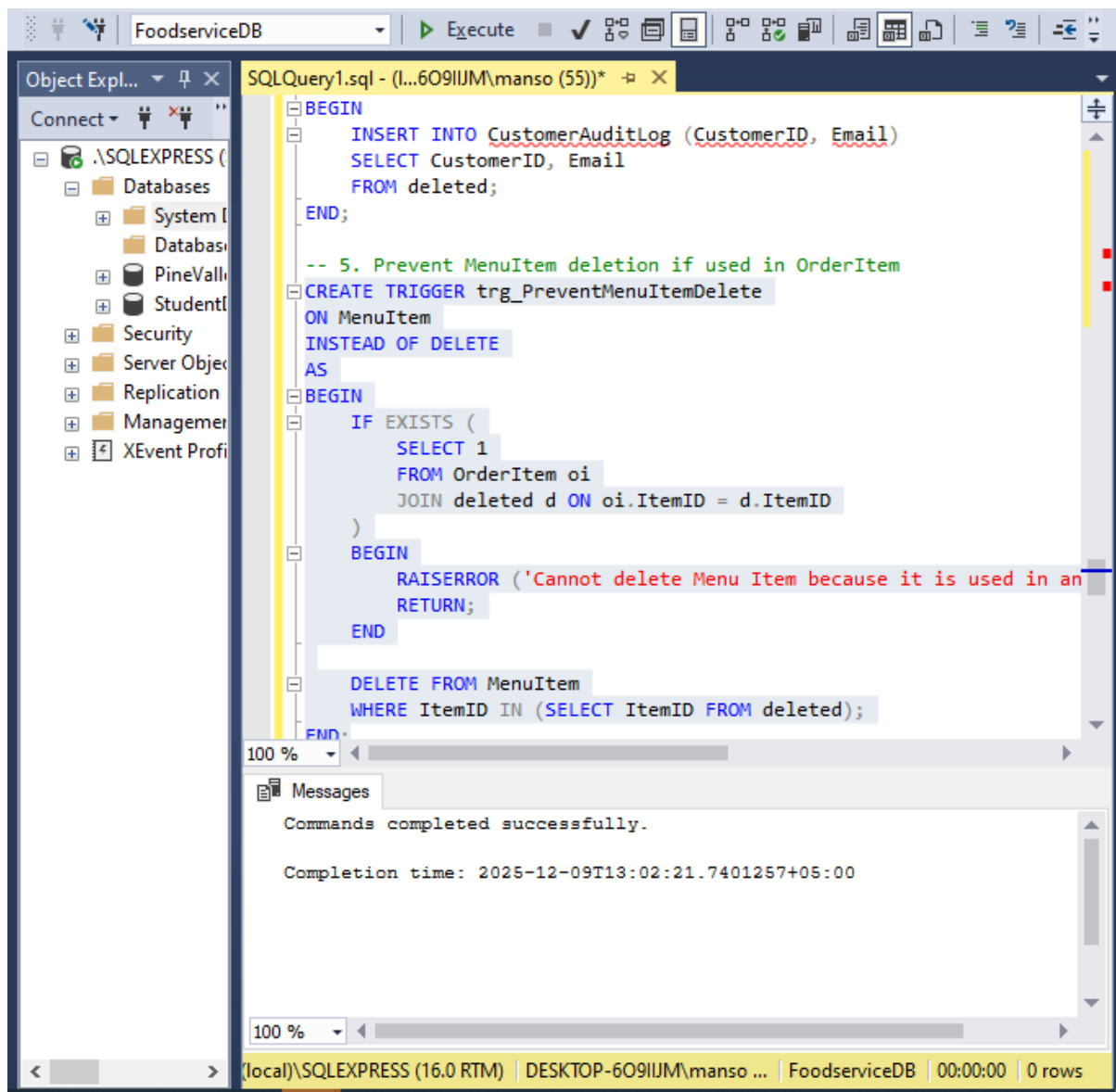
```
CREATE TRIGGER trg_LogCustomerDeletion
ON Customer
AFTER DELETE
AS
BEGIN
    INSERT INTO CustomerAuditLog (CustomerID, Email)
    SELECT CustomerID, Email
    FROM deleted;
END;
```



5.Prevent MenuItem deletion if used in OrderItem

- Uses **INSTEAD OF DELETE** to check if a **MenuItem** is currently referenced in any existing **OrderItem** record before proceeding with deletion.
- Upholds historical data integrity by preventing the removal of items that have been part of completed transactions, ensuring past order details remain accurate.

```
CREATE TRIGGER trg_PreventMenuItemDelete
ON MenuItem
INSTEAD OF DELETE
AS
BEGIN
    IF EXISTS (
        SELECT 1
        FROM OrderItem oi
        JOIN deleted d ON oi.ItemID = d.ItemID
    )
    BEGIN
        RAISERROR ('Cannot delete Menu Item because it is used in an Order.', 16, 1);
        RETURN;
    END
    DELETE FROM MenuItem
    WHERE ItemID IN (SELECT ItemID FROM deleted);
END;
```



Transactions

1.Update Order Status & Change Rider Availability Together

- Executes two sequential status updates—marking the order as 'Delivered' and the rider as 'Available'—within one transaction.
- Enforces **Consistency** by guaranteeing that the Rider's status is only updated *after* the Order is confirmed delivered, preventing logical business errors.

BEGIN TRY

BEGIN TRANSACTION;

-- Mark Order as Delivered

UPDATE [Order]

SET Status = 'Delivered'

WHERE OrderID = 6001; -- replace with actual OrderID

-- Make Rider available

UPDATE Rider

SET Availability = 1

WHERE RiderID = 5001; -- replace with assigned RiderID

COMMIT TRANSACTION;

PRINT 'Order delivered successfully.';

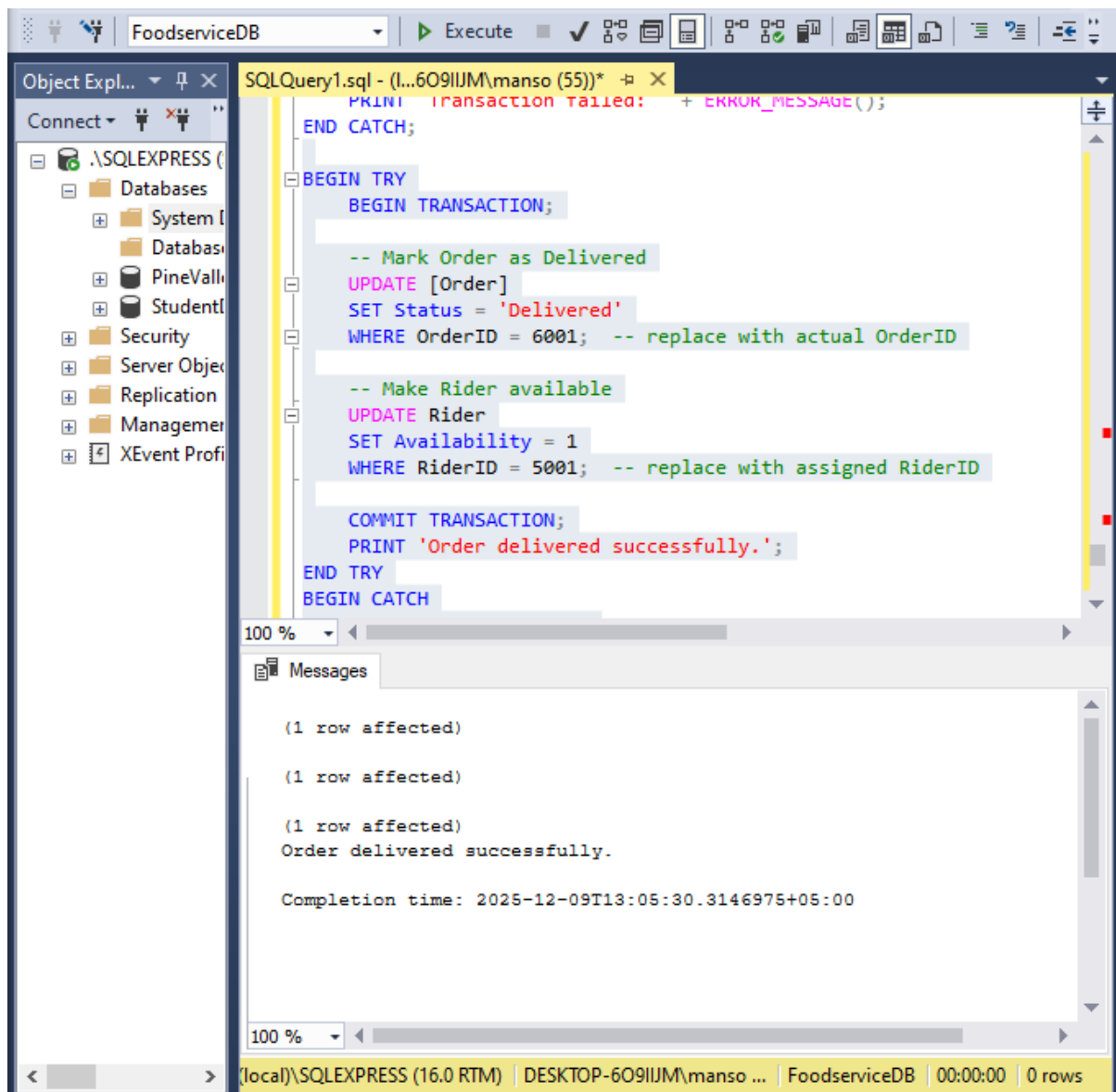
END TRY

BEGIN CATCH

ROLLBACK TRANSACTION;

PRINT 'Transaction failed: ' + ERROR_MESSAGE();

END CATCH;



2. Cancel Order + Delete Payment + Delete Items

- A controlled cleanup process that deletes dependent records (Payment and OrderItem) *before* deleting the main Order record.
- Ensures Referential Integrity during cancellation by safely preventing the creation of orphaned payment or order item records.

BEGIN TRY

BEGIN TRANSACTION;

-- Delete related Payment

DELETE FROM Payment

```
WHERE OrderID = 6001; -- replace with the OrderID to cancel
```

```
-- Delete related Order Items
```

```
DELETE FROM OrderItem
```

```
WHERE OrderID = 6001;
```

```
-- Delete the Order itself
```

```
DELETE FROM [Order]
```

```
WHERE OrderID = 6001;
```

```
COMMIT TRANSACTION;
```

```
PRINT 'Order canceled successfully.';
```

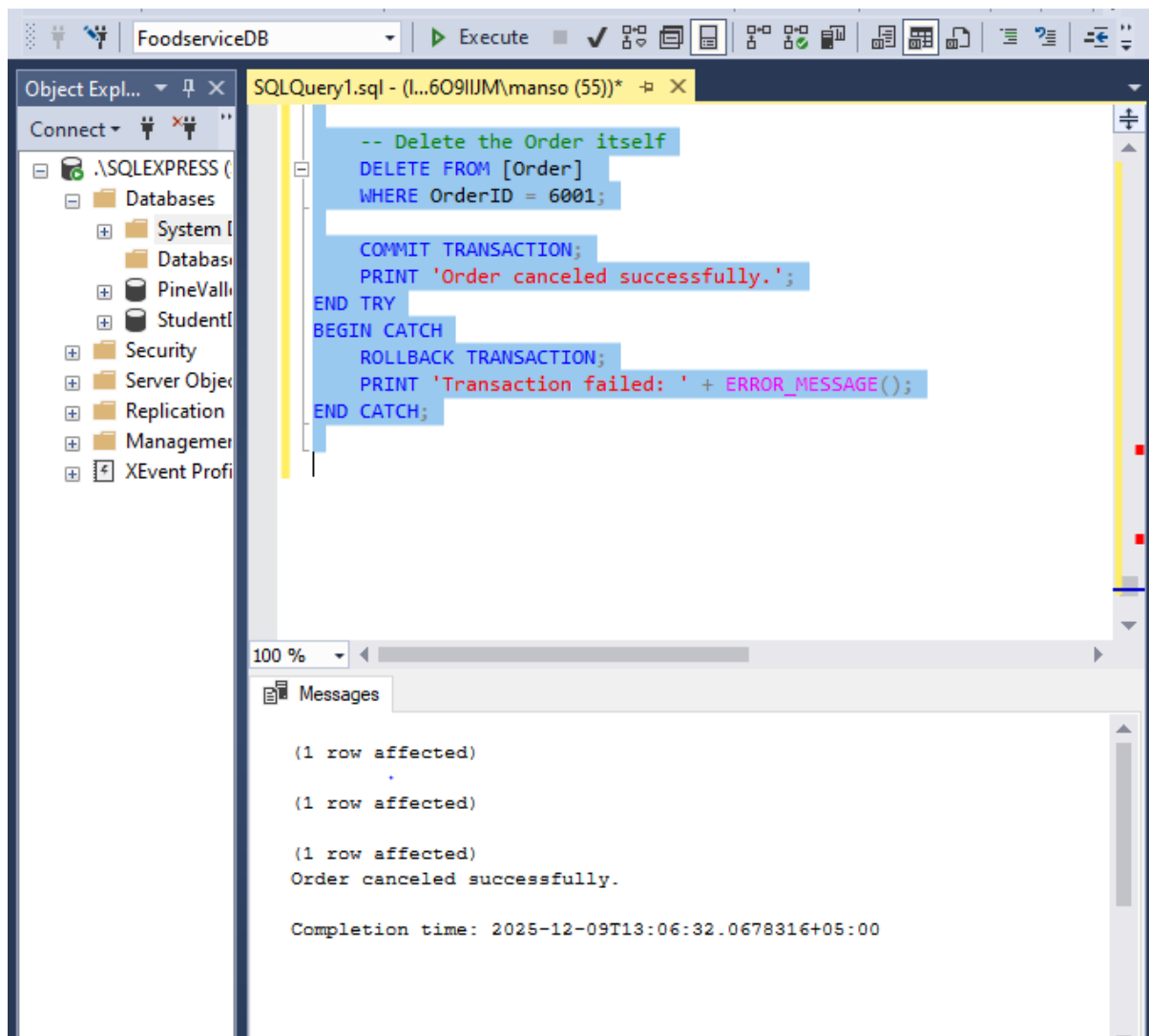
```
END TRY
```

```
BEGIN CATCH
```

```
ROLLBACK TRANSACTION;
```

```
PRINT 'Transaction failed: ' + ERROR_MESSAGE();
```

```
END CATCH;
```



ROLES

Admin_Role

- The **Admin_Role** is granted **full administrative access** (SELECT, INSERT, UPDATE, DELETE) across **all** seven core database tables to provide complete system management and oversight.

```
CREATE ROLE Admin_Role;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON Customer TO Admin_Role;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON Restaurant TO Admin_Role;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON MenuItem TO Admin_Role;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON Rider TO Admin_Role;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON [Order] TO Admin_Role;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON OrderItem TO Admin_Role;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON Payment TO Admin_Role;
```

Restaurant_Manager_Role

- Grants the necessary permissions (SELECT, INSERT, UPDATE) to manage the **menu items** and update the **restaurant's details**.

```
CREATE ROLE Restaurant_Manager_Role;
```

```
GRANT SELECT, INSERT, UPDATE ON MenuItem TO Restaurant_Manager_Role;
```

```
GRANT SELECT, UPDATE ON Restaurant TO Restaurant_Manager_Role;
```

Rider_Role

- Provides minimal access to view assigned **orders** and update the rider's personal **availability status**.

```
CREATE ROLE Rider_Role;
```

GRANT SELECT ON [Order] TO Rider_Role;

GRANT UPDATE ON Rider TO Rider_Role;

Customer_Role

- Allows the end-user to browse **restaurants** and **menu items** and track the status of their own placed **orders**.

CREATE ROLE Customer_Role;

GRANT SELECT ON Restaurant TO Customer_Role;

GRANT SELECT ON MenuItem TO Customer_Role;

GRANT SELECT ON [Order] TO Customer_Role;